
DANCE: Dynamic, Available, Neighbor-gated Condensation for Federated Text-Attributed Graphs

Zekai Chen^{1*} Haodong Lu^{1*} Xunkai Li¹ Henan Sun² Jia Li² Hongchao Qin¹ Rong-Hua Li¹
Guoren Wang¹

Abstract

Federated graph learning (FGL) enables collaborative training on graph data across multiple clients. With the rise of large language models (LLMs), textual attributes in FGL graphs are gaining attention. Text-attributed graph federated learning (TAG-FGL) improves FGL by explicitly leveraging LLMs to process and integrate these textual features. However, current TAG-FGL methods face three main challenges: **(1) Overhead.** LLMs for processing long texts incur high costs. To make TAG-FGL practical, graph condensation (GC) offers a direct way to reduce computation load, but applying it to TAG-FGL also brings new challenges. **(2) Suboptimal.** GC-based TAG-FGL typically performs one-shot condensation with fixed LLM surrogates, which can become mismatched with the evolving global model, leading to suboptimal performance. **(3) Interpretability.** LLM-based condensation further introduces a black-box bottleneck: summaries lack faithful attribution and clear grounding to specific source spans, making local inspection and auditing difficult. To address the above issues, we propose **DANCE**, a new TAG-FGL paradigm with GC. To improve suboptimal performance, DANCE performs model-in-the-loop condensation refresh as the global model evolves. To enhance interpretability, DANCE preserves provenance by storing locally inspectable evidence packs that trace predictions to selected neighbors and source text spans. Across 8 TAG datasets, DANCE improves accuracy by **2.33%** at an **8%** condensation ratio, with **33.42%** fewer tokens per condensed node than TAG-FGL baselines.

^{*}Equal contribution ¹Beijing Institute of Technology, Beijing, China ²The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China. Correspondence to: Rong-Hua Li <lronghuabit@126.com>.

1. Introduction

Federated graph learning (FGL) enables collaborative training on graph data across multiple clients while preserving privacy by avoiding the sharing of raw data. With the rise of large language models (LLMs), the role of textual attributes in graph data has gained increasing attention (Wu et al., 2022; Qiu, 2023; Qu et al., 2023). Text-attributed graph federated learning (TAG-FGL) improves the FGL framework by explicitly leveraging LLMs to process and integrate these textual features, enabling the learning process on text-rich graphs. This integration enhances FGL’s ability to leverage not only the graph’s structure but also the rich semantic information contained in node texts.

Recent approaches to TAG-FGL integrate language models in different ways, e.g., via shared global modules or federated backbones (Wu et al., 2025b; Zhu et al., 2025), or by client-side LLM-based augmentation before standard subgraph-FL optimization (Yan et al., 2025). However, *practical* TAG-FGL still faces three challenges. **(1) Overhead.** Invoking LLMs to process long node texts incurs substantial token and computation costs, which can slow optimization under tight compute budget (Yan et al., 2025; Wu et al., 2025b; Zhu et al., 2025). To mitigate this overhead, we introduce **graph condensation (GC)** into TAG-FGL, aiming to reduce the amount of text and neighborhood information that must be processed. **(2) Suboptimal.** GC-based TAG-FGL typically performs one-shot condensation with *fixed* LLM surrogates, such as summaries or embeddings. Since these surrogates are generated before or outside the evolving federated optimization process, they can become mismatched with the latest global model, leading to suboptimal performance. **(3) Interpretability.** LLM-based condensation can further create a black-box bottleneck: the generated summaries and embeddings often lack faithful attribution and clear grounding to specific source spans, making it difficult to verify which original texts, selected neighbors, or evidence fragments support a prediction. This limits local inspection and auditing in regulated settings (Lopez-Ramos et al., 2024; Saifullah et al., 2024).

To address the above issues, we propose **DANCE** (**D**ynamic, **A**vailable, **N**ighbor-gated **C**ondensation), a new TAG-FGL

paradigm built on GC. To improve **suboptimal** performance caused by one-shot condensation, DANCE performs round-wise condensation refresh using the latest global model. To enhance **interpretability**, DANCE preserves provenance by keeping locally inspectable evidence packs that trace predictions back to selected neighbors and source text spans. Concretely, at each refresh round, each client executes three steps in sequence: (1) *Label-aware node condensation* selects a condensed core as the candidate node set for GC; (2) *Hierarchical text condensation* selects evidence-providing neighbors and distills a compact set of readable summaries/evidence spans, while recording the selected neighbors and source spans as local evidence packs; (3) *Self-expressive topology reconstruction* rebuilds a lightweight propagation graph on the condensed core using fused graph-text representations to restore connectivity for downstream training.

Contributions. (1) *New Formulation.* We introduce GC for TAG-FGL, which integrates interpretable LLM-based summary generation, dynamically adjusted to the evolving global model. (2) *New Framework.* We present DANCE, a novel TAG-FGL framework that combines label-aware node condensation, hierarchical text condensation, and self-expressive topology reconstruction, enhancing both performance and interpretability. (3) *SOTA Performance.* On 8 TAG datasets, DANCE achieves SOTA performance, surpassing competitive TAG-FGL baselines by 2.33% on accuracy while reducing token processing costs by 33.42% per condensed node.

2. Preliminaries

Text-Attributed Graphs. A text-attributed graph (TAG) is $G = (V, A, S)$, where V is the node set, $A \in \{0, 1\}^{|V| \times |V|}$ is the adjacency matrix, and $S = \{s_v\}_{v \in V}$ denotes node texts. A text encoder maps each s_v to a d -dimensional embedding:

$$x_v = \text{Enc}_\psi(s_v) \in \mathbb{R}^d, X = [x_v]_{v \in V} \in \mathbb{R}^{|V| \times d}. \quad (1)$$

We study semi-supervised node classification, where a labeled subset $V_L \subset V$ is associated with labels $\{y_v\}_{v \in V_L}$.

Graph Condensation for TAGs. Graph condensation constructs a compact surrogate TAG $\hat{G}$ that preserves task-relevant information of G while reducing cost. A condensed TAG is $\hat{G} = (\hat{V}, \hat{A}, \hat{S})$, where $|\hat{V}| \ll |V|$, $\hat{S} = \{\hat{s}_v\}_{v \in \hat{V}}$ denotes condensed node texts, and $\hat{A} \in \mathbb{R}^{|\hat{V}| \times |\hat{V}|}$ is a learned/reconstructed adjacency matrix (typically weighted and sparse). We train the downstream predictor on the condensed TAG and aim to retain comparable node classification performance to training on the original TAG.

Federated Learning on TAGs. The global TAG is distributed across M clients; client m holds a local subgraph

$G^{(m)} = (V^{(m)}, A^{(m)}, S^{(m)})$. At round t , the server broadcasts global model $\omega^{(t-1)}$ to clients $\mathcal{M}_t$. Each client optimizes locally and uploads an update $\Delta\omega_m^{(t)}$; the server aggregates

$$\omega^{(t)} = \text{Agg}\left(\{\Delta\omega_m^{(t)}\}_{m \in \mathcal{M}_t}\right), \quad (2)$$

where Agg is an aggregation method. Our goal is to learn global model ω for semi-supervised node classification while keeping raw texts/graphs local in the FL setting.

3. Related Work

Graph neural networks. Graph neural networks (GNNs) learn node representations via neighborhood propagation and aggregation, serving as standard backbones for semi-supervised node classification (Kipf & Welling, 2017; Hamilton et al., 2017; Veličković et al., 2018). Later variants improve robustness on heterophily and structural heterogeneity via geometric encodings and heterophily-aware designs (Pei et al., 2020; Chien et al., 2021; Zhu et al., 2020). In our setting, these models mainly serve as structural encoders that must be coupled with rich text attributes.

Federated graph learning. FGL extends federated learning to graph-structured data, typically under a subgraph-FL protocol where each client holds a local subgraph and collaboratively trains a global GNN without sharing raw graphs (Fu et al., 2022; He et al., 2021; Wu et al., 2025a). Prior work addresses non-IIDness via structural repair, personalization, and topology-aware aggregation, e.g., missing-neighbor generation (Zhang et al., 2021), personalized heads (Baek et al., 2023), topology-aware confidence (Li et al., 2023), and condensation-based collaboration (Chen et al., 2025). However, most methods assume compact node features; when attributes are long texts, *what* evidence to encode/propagate (and under what budgets) becomes a first-order bottleneck that is less explored.

Federated learning on TAGs. As FGL enters text-rich domains, existing TAG-FGL methods incorporate language signals at different layers. (1) *Client-side augmentation* uses LLMs to enrich local texts/neighborhoods before subgraph-FL (Yan et al., 2025). (2) *Federated modular coordination* shares global modules (e.g., codebooks/prompts/prototypes) to align topology-text learning without raw-text transmission (Wu et al., 2025b; Zhu et al., 2025). (3) *Semantic-prior injection* regularizes message passing using label/ontology semantics or external knowledge. Despite these advances, most methods treat text as static embeddings or absorb it into a monolithic backbone, while *budgeted, round-adaptive* control of neighbor usage and evidence retention is not a primary design focus. DANCE complements them by making evidence and neighbor selection round-adaptive under explicit budgets and FL constraints, while keeping human-readable traces locally available.

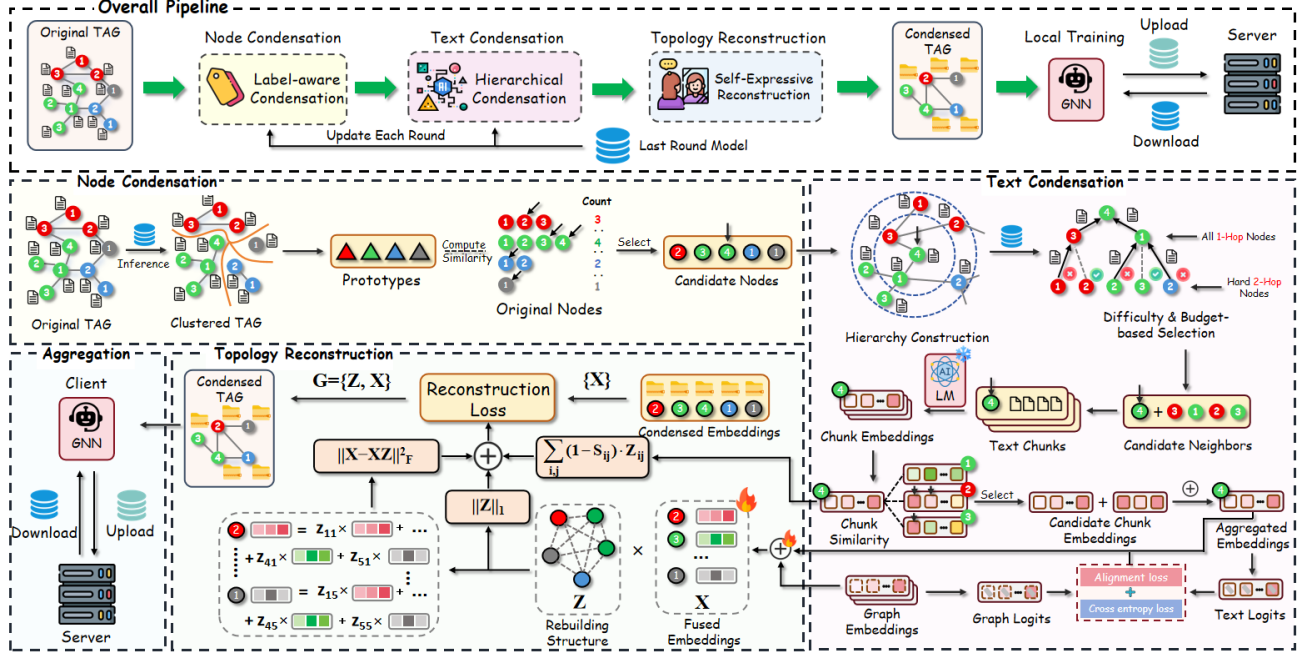


Figure 1. Overview of the proposed **DANCE** framework. Each client condenses a local TAG by (1) label-aware node condensation and (2) budgeted hierarchical text propagation/condensation, then (3) rebuilds a lightweight topology via self-expressive reconstruction on fused graph-text embeddings and (4) trains a GNN for federated aggregation.

4. Methodology

4.1. Overview

In this section, we present a detailed description of **DANCE**. We introduce its three core modules: (1) Label-aware node condensation, (2) Hierarchical text condensation, and (3) Self-expressive topology reconstruction. We then summarize the complete round-wise client-server workflow in Alg. 1, with an overview of the framework shown in Fig. 1.

4.2. Label-aware Node Condensation

Motivation. TAG-FGL is often bottlenecked by long-text processing and multi-hop propagation. We therefore perform label-aware clustering-based node condensation to reduce computation and communication overhead. We perform node condensation every 10 communication rounds.

Model-conditioned pseudo-labels. At round t , client m runs the last-round global model $\omega^{(t-1)}$ once on its local TAG to obtain node embeddings $\{z_v\}_{v \in V^{(m)}}$ and predicted labels $\hat{y}_v$, and their confidence scores π_v (reusing cached text embeddings when available). We assign pseudo-labels as follows:

$$\tilde{y}_v = \begin{cases} y_v, & v \in V_L^{(m)}, \\ \hat{y}_v, & v \notin V_L^{(m)} \text{ and } \pi_v \geq \tau, \\ \text{unassigned}, & v \notin V_L^{(m)} \text{ and } \pi_v < \tau. \end{cases} \quad (3)$$

We use τ to keep only high-confidence pseudo-labels ($\pi_v \geq$

τ) when estimating class statistics.

Distribution-preserving core selection. For each class c , we obtain $P_c = \min(P, |V_c^{(m)}|)$ prototypes $\mathcal{P}_c$ by mini-batch k -means on $\{z_v\}_{v \in V_c^{(m)}}$. Each node v is scored by its best prototype match $s_v = \max_{p \in \mathcal{P}_{\tilde{y}_v}} \kappa(z_v, p)$ (cosine similarity). Given $r \in (0, 1]$, we set $K = \lceil r|V^{(m)}| \rceil$ and allocate class quotas proportional to the confident pseudo-label distribution; we then select top- s_v nodes per class to form $\hat{V}_t^{(m)}$.

4.3. Hierarchical Text Condensation

Motivation. Even on the condensed core $\hat{V}_t^{(m)}$, aggregating all multi-hop texts is costly and often redundant. We thus perform budgeted evidence aggregation: (i) select evidence-providing neighbors under per-hop budgets, and (ii) distill text chunks under token budget to form a compact evidence embedding for each core node.

Frozen text bank and caching. For each node $u \in V^{(m)}$, we split its node text into short chunks by sentence $s_{u,r}$ and encode each chunk with a frozen text encoder $e_{u,r} = \text{Enc}(s_{u,r}) \in \mathbb{R}^d$, and pool chunk embeddings to obtain a node-level text embedding $t_u = \text{p}(\{e_{u,r}\}_{r=1}^{R_u}) \in \mathbb{R}^d$. Here, $\text{p}(\cdot)$ denotes softmax-based attention pooling. Since Enc is fixed, $\{t_u\}$ can be cached across rounds. At round t , we obtain graph-side embeddings $\{g_v\}$ by a forward pass of the last-round graph encoder on the *original* local TAG.

Budgeted neighbor gating. For a core node $v \in \hat{V}_t^{(m)}$, let $\mathcal{N}_v^{(\ell)}$ be its ℓ -hop neighbors in the original local graph ($\ell \in \{0, 1, 2\}$). We score candidates in $\tilde{\mathcal{N}}_v^{(\ell)} (\tilde{\mathcal{N}}_v^{(0)} = \{v\})$ and select a sparse subset under hop budgets B_ℓ (Eq. (4)–(5)). Given the significantly larger size of the 2-hop neighborhood, we additionally pre-filter $\mathcal{N}_v^{(2)}$ with difficulty score u_v (Eq. (21)).

We score a candidate neighbor $u \in \tilde{\mathcal{N}}_v^{(\ell)}$ by cross-modal attention score between g_v and t_u :

$$s_{v,u} = \frac{(W_q g_v)^\top (W_k t_u)}{\sqrt{d}}. \quad (4)$$

For each hop $\ell \in \{0, 1, 2\}$, we transform $\{s_{v,u}\}_{u \in \tilde{\mathcal{N}}_v^{(\ell)}}$ into hop-wise sparse attention weights $\alpha_{v,\cdot}^{(\ell)}$ under budget B_ℓ :

$$\alpha_{v,\cdot}^{(\ell)} = \Pi_{B_\ell}(\text{entmax}(\{s_{v,u}\}_{u \in \tilde{\mathcal{N}}_v^{(\ell)}})), \quad (5)$$

where $\text{entmax}(\cdot)$ yields sparse attention weights, selecting at most B_ℓ neighbors (implemented with a straight-through estimator). We denote the selected nodes at hop ℓ as $\mathcal{S}_v^{(\ell)} = \{u \in \tilde{\mathcal{N}}_v^{(\ell)} : \alpha_{v,u}^{(\ell)} > 0\}$. Then we aggregate the selected multi-hop evidence into a hierarchical context vector c_v :

$$c_v = \sum_{\ell \in \{0,1,2\}} \gamma_\ell \sum_{u \in \mathcal{S}_v^{(\ell)}} \alpha_{v,u}^{(\ell)} t_u, \quad \gamma_\ell \geq 0, \quad \sum_\ell \gamma_\ell = 1. \quad (6)$$

Chunk selection. Neighbor gating determines *which nodes* can provide evidence; we also select *which chunks* to retain in the token budget B_{tok} . We collect chunk candidates only from the selected neighbors and denote the resulting candidate set as $\mathcal{E}_v$. Specifically, we form an attention query from the core-node graph embedding, $q_v = W_s g_v$, compute chunk-level attention scores $a_{v,(u,r)}$, and obtain budgeted sparse weights $\pi_{v,\cdot}$ via $\Pi_{B_{\text{tok}}}(\cdot)$:

$$a_{v,(u,r)} = \frac{q_v^\top e_{u,r}}{\sqrt{d}}, \quad (7)$$

$$\pi_{v,\cdot} = \Pi_{B_{\text{tok}}}(\text{entmax}(\{a_{v,(u,r)}\}_{(u,r) \in \mathcal{E}_v})),$$

where $\text{entmax}(\cdot)$ yields sparse attention weights, selecting at most B_{tok} chunks (implemented with a straight-through estimator). We aggregate selected chunks into a compact evidence embedding:

$$\tilde{t}_v = \sum_{(u,r) \in \mathcal{E}_v} \pi_{v,(u,r)} e_{u,r}. \quad (8)$$

By construction, each core node uses at most $\sum_\ell B_\ell$ neighbors and at most B_{tok} chunks. Gradients do not backpropagate into the frozen encoder Enc ; we update only the graph backbone and selection parameters (e.g., W_q, W_k, W_s, γ).

Local evidence traces and summarization. For each core node, we store the top gated neighbors (via $\alpha_{v,\cdot}^{(\ell)}$) and selected evidence chunks (via $\pi_{v,\cdot}$), which can be mapped

back to the original texts. We then prompt an on-device LLM to summarize the selected chunks into short evidence summaries, which can be cached and reused across rounds. The prompt template is provided in Appendix C.4. Neither raw texts, audit traces (neighbor/chunk selections and weights), nor the on-device summaries are transmitted.

4.4. Self-expressive Topology Reconstruction

Motivation. After node/text condensation, many original edges are discarded, weakening message passing on the retained core. To restore propagation capacity *without* reverting to the full local graph, we learn a sparse self-expression coefficient matrix Z over $\hat{V}_t^{(m)}$ and synthesize a lightweight adjacency $\hat{A}_t^{(m)}$ from Z for efficient message passing on the condensed core.

Gated cross-modal fusion. For each $v \in \hat{V}_t^{(m)}$, we fuse the graph embedding g_v and condensed evidence embedding $\tilde{t}_v$ into a unified feature $x_v \in \mathbb{R}^d$:

$$\begin{aligned} \alpha_v &= \sigma(w^\top [g_v \parallel \tilde{t}_v]), \\ x_v &= \text{LN}(W_g g_v + \alpha_v W_t \tilde{t}_v). \end{aligned} \quad (9)$$

Stacking $\{x_v\}_{v \in \hat{V}_t^{(m)}}$ yields the fused feature matrix X . We learn $\{W_g, W_t, w\}$ with a fused-view supervised loss and a lightweight cross-view alignment regularizer. Here $\mathbb{E}_{v \in \mathcal{S}}[\cdot] \triangleq \frac{1}{|\mathcal{S}|} \sum_{v \in \mathcal{S}} (\cdot)$ denotes the empirical average over a node set $\mathcal{S}$. Let $p_v = \text{softmax}(o_v)$, $p_v^g = \text{softmax}(o_v^g)$, and $p_v^t = \text{softmax}(o_v^t)$.

$$\mathcal{L}_m = \mathbb{E}_{v \in \hat{V}_{t,L}^{(m)}} \text{CE}(p_v, y_v) + \lambda_{\text{align}} \mathbb{E}_{v \in \hat{V}_t^{(m)}} D(p_v^g, p_v^t). \quad (10)$$

where $o_v = \text{Dec}(x_v)$, $o_v^g = \text{Dec}_g(g_v)$, and $o_v^t = \text{Dec}_t(\tilde{t}_v)$ are logits from lightweight heads and $D(\cdot, \cdot)$ is a divergence (e.g., KL). The text encoder used to form $\tilde{t}_v$ remains frozen.

Prior-regularized sparse self-expression. Let $S_{ij} \in [0, 1]$ denote an evidence-induced prior measuring textual support between nodes i and j . For each core node $i \in \hat{V}_t^{(m)}$, we construct an evidence-aware candidate set $\mathcal{C}(i)$ by taking the union of two Top- q neighbor lists: (i) neighbors with the highest fused-feature similarity $x_i^\top x_j$, and (ii) neighbors with the strongest evidence prior S_{ij} . We then solve a sparse self-expression problem on the fused feature matrix X :

$$\min_Z \alpha \|X - XZ\|_F^2 + \beta \|Z\|_1 + \sum_{i \neq j} (1 - S_{ij}) |Z_{ij}|, \quad (11)$$

Here, the first term enforces reconstruction fidelity, the ℓ_1 penalty promotes sparsity, and the prior-regularizer suppresses coefficients on pairs with weak textual support. The hard constraint restricts each row of Z to $\mathcal{C}(i)$, yielding a lightweight evidence-guided reconstruction.

Sparsified adjacency synthesis. We symmetrize and sparsify the reconstructed coefficients to obtain the propagation graph:

$$\hat{A}_t^{(m)} = \mathcal{T}_k(|Z| + |Z|^\top), \quad (12)$$

which yields an $O(Kk)$ -edge graph for efficient message passing on the condensed core.

5. Theoretical Analysis

In this section, we provide a theoretical analysis of **DANCE** to justify its budgeted evidence design. We focus on three aspects: (i) hard budget guarantees and per-round cost decomposition, (ii) approximation error from hard budget projection, and (iii) stability of model-in-the-loop evidence refresh. All proofs are deferred to Appendix E.

5.1. Budget Guarantees and Cost Decomposition

The truncation operator $\Pi_B(\cdot)$ enforces a hard budget by retaining the top- B entries and renormalizing their weights.

Theorem 5.1 (End-to-end cost under hard budgets). *For client m , let $\hat{V}^{(m)}$ be the condensed core set with $|\hat{V}^{(m)}| = K$. For each core node $v \in \hat{V}^{(m)}$, DANCE selects at most B_ℓ neighbors at hop $\ell \in \{0, 1, 2\}$ and at most B_{tok} text chunks. Therefore,*

$$|\mathcal{S}_v^{(\ell)}| \leq B_\ell, \quad \|\pi_{v,\cdot}\|_0 \leq B_{\text{tok}}, \quad (13)$$

and

$$\sum_{v \in \hat{V}^{(m)}} \|\pi_{v,\cdot}\|_0 \leq KB_{\text{tok}}. \quad (14)$$

The per-round cost on client m is decomposed as

$$\text{Cost}^{(m)} = \text{Cost}_{\text{agg}}^{(m)} + \text{Cost}_{\text{backbone}}^{(m)} + \text{Cost}_{\text{sel}}^{(m)}, \quad (15)$$

where the three terms denote evidence aggregation, backbone encoding, and scoring/selection, respectively.

Let d be the embedding dimension. The budgeted aggregation cost is

$$\text{Cost}_{\text{agg}}^{(m)} = O\left(dK\left(\sum_{\ell \in \{0,1,2\}} B_\ell + B_{\text{tok}}\right)\right). \quad (16)$$

The scoring/selection cost and the backbone cost are additive and linear in the selected evidence size. Thus, the end-to-end per-round cost remains bounded by the hard budgets.

5.2. Approximation Error from Hard Budget Projection

We quantify the error introduced by hard truncation relative to using the same chunk-scoring rule without truncation.

Proposition 5.2 (Approximation error under hard budget projection). *For a probability vector $p \in \Delta^n$, let $T_B(p)$ be the indices of its top- B entries and define the discarded tail mass as*

$$\delta_B(p) \triangleq \sum_{i \notin T_B(p)} p_i. \quad (17)$$

For a core node v , let $p_v = \text{entmax}(\{a_{v,(u,r)}\}_{(u,r) \in \mathcal{E}_v})$ be the untruncated chunk-weight distribution, and let $\pi_v = \Pi_{B_{\text{tok}}}(p_v)$ be the budgeted distribution. Let t_v^{full} and $\tilde{t}_v$ denote the unbudgeted and budgeted evidence embeddings, respectively. Under Assumption E.1,

$$\|\tilde{t}_v - t_v^{\text{full}}\|_2 \leq 2M \cdot \delta_{B_{\text{tok}}}(p_v). \quad (18)$$

The same bound applies to neighbor gating at hop ℓ by replacing B_{tok} with B_ℓ .

This result shows that the truncation error is controlled by the discarded probability mass. When $\text{entmax}(\cdot)$ already produces sparse weights, hard truncation removes little useful evidence and introduces only a small error.

5.3. Stability of Model-in-the-loop Evidence Refresh

DANCE refreshes selected neighbors and chunks according to model-dependent scores. Since $\text{entmax}(\cdot)$ followed by top- B truncation preserves score ranking, the selected evidence changes only when the ranking near the top- B boundary changes.

Theorem 5.3 (Stability of DANCE evidence selection). *Let $r(\omega)$ be the pre-entmax score vector, and let $T_B(r(\omega))$ denote the indices of its top- B entries. Define the top- B margin as*

$$\Delta_B(\omega) \triangleq r_{(B)}(\omega) - r_{(B+1)}(\omega), \quad (19)$$

where $r_{(B)}(\omega)$ and $r_{(B+1)}(\omega)$ are the B -th and $(B+1)$ -th largest scores.

Under Assumption E.9, if

$$\|\omega - \omega'\|_2 \leq \frac{\Delta_B(\omega)}{2L_s}, \quad (20)$$

then the selected top- B index set remains unchanged after applying $\text{entmax}(\cdot)$ and Π_B . This holds for both neighbor and chunk selection, i.e., $B \in \{B_\ell, B_{\text{tok}}\}$.

6. Experiments

In this section, we provide a comprehensive empirical evaluation of the proposed **DANCE** framework by addressing the following key research questions:

Q1 – Does DANCE improve overall accuracy over strong FGL baselines across diverse benchmarks? **Q2** – How interpretable are the decision-making processes in DANCE,

Table 1. Node classification accuracy (%) comparison across datasets with varying condensation ratios. For methods with graph or data condensation, results are reported under the specified condensation ratios, while methods without condensation are evaluated on the full datasets. The highest results are highlighted in **bold**, while the second-highest in underline.

Setting			Methods											
Property	Dataset	Ratio	FedAvg	FedSage+	FedGTA	LLM4FGL	LLM4RGNN	LLaTa	GCond	SFGC	FedC4	FedGVD	FedGM	Ours
Small	Cora	8%							83.61 $\pm$ 0.26	83.58 $\pm$ 0.25	86.02 $\pm$ 0.21	85.56 $\pm$ 0.20	84.15 $\pm$ 0.13	88.87$\pm$0.43
		4%	82.45 $\pm$ 0.35	83.37 $\pm$ 0.54	84.65 $\pm$ 0.40	79.45 $\pm$ 0.23	86.73 $\pm$ 0.42	<u>87.26$\pm$0.32</u>	82.97 $\pm$ 0.40	83.15 $\pm$ 0.53	84.98 $\pm$ 0.42	83.72 $\pm$ 0.46	80.78 $\pm$ 0.41	87.12 $\pm$ 0.42
		2%							81.36 $\pm$ 0.61	81.42 $\pm$ 0.65	84.33 $\pm$ 0.56	83.24 $\pm$ 0.64	74.09 $\pm$ 0.54	86.32 $\pm$ 0.66
	CiteSeer	8%							72.03 $\pm$ 0.27	73.14 $\pm$ 0.24	75.95 $\pm$ 0.22	72.61 $\pm$ 0.27	73.82 $\pm$ 0.21	80.19$\pm$0.76
		4%	75.28 $\pm$ 0.41	75.12 $\pm$ 0.51	75.44 $\pm$ 0.51	75.53 $\pm$ 0.22	76.96 $\pm$ 0.46	<u>78.21$\pm$0.43</u>	67.46 $\pm$ 0.40	71.03 $\pm$ 0.45	74.84 $\pm$ 0.36	72.11 $\pm$ 0.51	72.57 $\pm$ 0.43	78.19 $\pm$ 0.48
		2%							67.27 $\pm$ 0.66	69.34 $\pm$ 0.65	73.91 $\pm$ 0.62	67.22 $\pm$ 0.57	70.21 $\pm$ 0.43	75.93 $\pm$ 0.63
Medium	Arxiv	8%							68.92 $\pm$ 0.31	70.48 $\pm$ 0.32	72.87 $\pm$ 0.24	70.08 $\pm$ 0.21	71.72 $\pm$ 0.23	78.26$\pm$0.26
		4%	72.19 $\pm$ 0.48	OOT	71.18 $\pm$ 0.41	69.59 $\pm$ 0.14	70.72 $\pm$ 0.35	73.24 $\pm$ 0.44	67.23 $\pm$ 0.41	69.24 $\pm$ 0.51	71.44 $\pm$ 0.36	69.07 $\pm$ 0.44	69.83 $\pm$ 0.56	75.15 $\pm$ 0.46
		2%							65.47 $\pm$ 0.57	67.24 $\pm$ 0.61	69.83 $\pm$ 0.57	67.22 $\pm$ 0.60	68.34 $\pm$ 0.59	73.36 $\pm$ 0.56
	WikiCS	8%							79.33 $\pm$ 0.25	80.08 $\pm$ 0.33	81.27 $\pm$ 0.21	79.14 $\pm$ 0.31	79.92 $\pm$ 0.30	84.26$\pm$0.35
		4%	76.26 $\pm$ 0.43	81.25 $\pm$ 0.39	79.35 $\pm$ 0.54	79.31 $\pm$ 0.18	OOT	<u>81.58$\pm$0.39</u>	78.04 $\pm$ 0.53	78.97 $\pm$ 0.48	80.63 $\pm$ 0.43	78.71 $\pm$ 0.42	79.02 $\pm$ 0.52	81.27 $\pm$ 0.42
		2%							76.47 $\pm$ 0.62	77.34 $\pm$ 0.68	79.36 $\pm$ 0.66	76.99 $\pm$ 0.57	78.43 $\pm$ 0.49	80.58 $\pm$ 0.56
Inductive	Instagram	2%							64.82 $\pm$ 0.23	65.12 $\pm$ 0.34	64.83 $\pm$ 0.28	63.52 $\pm$ 0.26	63.57 $\pm$ 0.21	65.17$\pm$0.22
		1%	64.15 $\pm$ 0.51	64.07 $\pm$ 0.49	64.12 $\pm$ 0.53	60.98 $\pm$ 0.32	OOT	64.53 $\pm$ 0.55	64.38 $\pm$ 0.53	64.12 $\pm$ 0.44	64.03 $\pm$ 0.42	62.64 $\pm$ 0.44	64.25 $\pm$ 0.40	64.84 $\pm$ 0.58
		0.5%							65.84 $\pm$ 0.58	63.14 $\pm$ 0.63	63.55 $\pm$ 0.58	62.24 $\pm$ 0.60	63.09 $\pm$ 0.46	64.75 $\pm$ 0.56
	Reddit	8%							67.23 $\pm$ 0.27	67.96 $\pm$ 0.20	66.08 $\pm$ 0.31	64.66 $\pm$ 0.20	65.73 $\pm$ 0.45	68.45$\pm$0.32
		4%	59.22 $\pm$ 0.45	63.28 $\pm$ 0.39	65.54 $\pm$ 0.48	64.32 $\pm$ 0.45	OOT	67.62 $\pm$ 0.54	65.76 $\pm$ 0.47	66.53 $\pm$ 0.50	65.24 $\pm$ 0.42	63.65 $\pm$ 0.50	64.45 $\pm$ 0.49	67.88 $\pm$ 0.41
		2%							63.94 $\pm$ 0.57	65.48 $\pm$ 0.62	64.35 $\pm$ 0.64	63.26 $\pm$ 0.56	63.54 $\pm$ 0.51	65.96 $\pm$ 0.62
Large	Children	1%				OOT	OOT	47.26 $\pm$ 0.38	47.32 $\pm$ 0.35	46.83 $\pm$ 0.21	48.24 $\pm$ 0.35	OOT	47.36 $\pm$ 0.40	48.25$\pm$0.28
		0.6%	42.02 $\pm$ 0.27	47.38 $\pm$ 0.30	47.16 $\pm$ 0.36				44.87 $\pm$ 0.27	45.14 $\pm$ 0.31	47.88 $\pm$ 0.37	OOT	45.02 $\pm$ 0.28	47.17 $\pm$ 0.35
		0.4%							42.54 $\pm$ 0.43	43.13 $\pm$ 0.54	47.53 $\pm$ 0.52	OOT	43.42 $\pm$ 0.30	46.18 $\pm$ 0.46
Hetero.	Ratings	8%							43.03 $\pm$ 0.63	44.02 $\pm$ 0.57	45.24 $\pm$ 0.70	44.93 $\pm$ 0.66	44.63 $\pm$ 0.62	47.84$\pm$0.61
		6%	40.75 $\pm$ 0.62	45.98 $\pm$ 0.66	43.56 $\pm$ 0.61	42.23 $\pm$ 0.52	OOT	44.47 $\pm$ 0.66	42.12 $\pm$ 0.70	42.97 $\pm$ 0.60	44.13 $\pm$ 0.56	43.85 $\pm$ 0.58	44.02 $\pm$ 0.61	46.88 $\pm$ 0.70
		4%							41.23 $\pm$ 0.63	42.14 $\pm$ 0.57	43.18 $\pm$ 0.71	43.41 $\pm$ 0.66	43.07 $\pm$ 0.68	45.43 $\pm$ 0.61

and how does it provide human-readable evidence for regulatory compliance? **Q3** – How robust is DANCE to hyperparameter choices, Non-IID data, and the number of clients? **Q4** – What efficiency gains does DANCE achieve in terms of overall cost compared to baseline federated graph learning methods? **Q5** – Can DANCE be combined with standard privacy mechanisms while maintaining competitive performance and privacy protection?

6.1. Experiments Setup

Baselines and Evaluation Metrics. Following prior studies on federated graph learning, we benchmark DANCE against representative methods from four categories. (1) FL/FGL includes FedAvg (McMahan et al., 2017), FedSage+ (Zhang et al., 2021), and FedGTA (Li et al., 2023), which represent strong FL and structure-aware FGL methods under subgraph-based FL settings. (2) Graph condensation includes GCond (Jin et al., 2021) and SFGC (Zheng et al., 2023) (3) Federated graph condensation includes FedC4 (Chen et al., 2025), FedGVD (Dai et al., 2025), and FedGM (Zhang et al., 2025a), which perform condensation without preserving readable textual attributes. (4) LLM-based TAG-FGL includes LLM4RGNN (Zhang et al., 2025c), LLaTA (Zhang et al., 2025b), and LLM4FGL (Yan et al., 2025), which leverage large language models to enhance or augment text-attributed graphs. Detailed descriptions of each baseline and the rationale for their inclusion are provided in Appendix C.12.1.

Datasets and Federated Simulation. We evaluate DANCE on text-attributed graph (TAG) benchmarks spanning graph

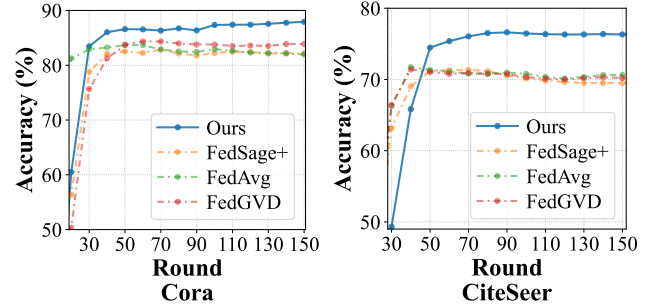


Figure 2. Convergence across communication round

scales and multiple domains. Additionally, we include heterophilic dataset to validate DANCE under graph heterogeneity. We follow the standard subgraph-FL protocol (Zhang et al., 2021) to partition each global TAG into client subgraphs (e.g., Louvain). Dataset statistics and default settings are reported in Table 7.

6.2. Overall Performance (Q1)

Overall Results. To answer Q1, Table 1 compares DANCE with all baselines across benchmarks. DANCE consistently achieves the best or second-best Accuracy across benchmarks, even under compact condensation settings, suggesting that the proposed framework can improve predictive performance without merely trading accuracy for efficiency. We further report convergence in Figure 2, showing that DANCE reaches competitive performance earlier in training. Detailed experimental settings and implementation specifics are provided in Appendix C.2.

Ablation Study. To better understand the source of perfor-

Table 2. Ablation study on Cora and CiteSeer datasets. Each variant removes one core module to validate its contribution. Bold values indicate the best performance in each column.

Datasets	Cora(0.08)		CiteSeer(0.08)	
	Overall-F1	Acc	Overall-F1	Acc
Full	88.36$\pm$0.52	88.87$\pm$0.43	77.16$\pm$0.83	80.19$\pm$0.76
w/o TGR	86.32 $\pm$ 0.64	87.08 $\pm$ 0.52	73.32 $\pm$ 0.88	77.32 $\pm$ 0.82
w/o TC	82.92 $\pm$ 0.71	85.14 $\pm$ 0.61	73.07 $\pm$ 1.02	77.11 $\pm$ 0.94
w/o NC	81.04 $\pm$ 0.68	81.22 $\pm$ 0.56	65.44 $\pm$ 0.95	75.19 $\pm$ 0.89

mance gains in DANCE, we conduct an ablation study by removing each core component in turn. As shown in Table 2, removing any core component leads to a consistent performance degradation across datasets. In particular, removing node condensation causes the largest drop, highlighting the importance of jointly condensing structure and text under federated non-IID settings.

6.3. Interpretability and Auditability Analysis (Q2)

To answer Q2, we conduct a qualitative case study and a complementary quantitative analysis to evaluate the interpretability of DANCE’s decision-making process and the human-readability of its extracted evidence, using faithfulness, sufficiency, and compactness as evaluation criteria.

Qualitative Case Studies. In Appendix C.5, we present qualitative case studies showing how DANCE produces concise, human-readable evidence packs and neighbor-level contribution traces. The case studies further validate faithful evidence selection via paired sufficiency and comprehensiveness analyses, enabling compliance-oriented inspection while respecting federated privacy constraints.

Faithfulness, sufficiency, and compactness. We evaluate chunk-level *faithfulness* via paired *sufficiency* and *comprehensiveness* tests following standard practice (Appendix C.5). For sufficiency, we re-predict using only the DANCE-selected chunks under the same test-time budget; for comprehensiveness, we remove the selected chunks while keeping all remaining text intact. As shown in Appendix C.5, retaining only the selected chunks largely preserves accuracy, whereas removing them causes a noticeably larger drop than budget-matched random deletion.

6.4. Robustness Analysis (Q3)

To answer Q3, we evaluate the robustness of DANCE under varying hyper-parameters, federation scales, and model refresh strategies, highlighting its robustness under different experimental conditions. Regarding Non-IID data, clients are constructed via Louvain community detection, which inherently induces structural and label heterogeneity across clients; thus, all experiments are conducted under a Non-IID federation setting.

Hyper-parameter Sensitivity. We examine the sensitiv-

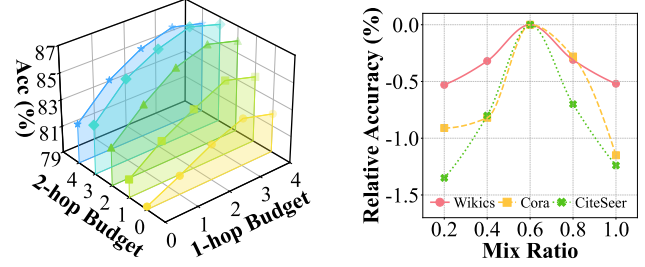


Figure 3. Hyper-parameter analysis of the *Hierarchical Text Condensation* module. **Left:** accuracy under different hop budgets used in aggregation. **Right:** relative accuracy w.r.t. the summary mixing ratio mix

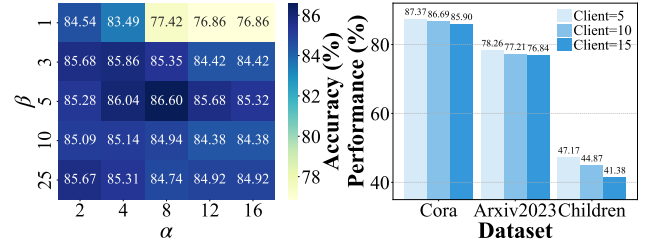


Figure 4. Hyper-parameter sensitivity of the *Self-expressive Topology Reconstruction* module and scalability analysis. **Left:** accuracy under different regularization weights α and β . **Right:** scalability with varying numbers of clients.

ity of DANCE to the key hyper-parameters in *Hierarchical Text Condensation* and *Self-expressive Topology Reconstruction*. As shown in Fig. 3, DANCE remains stable across a wide range of 1-hop/2-hop evidence budgets and the summary mixing ratio mix , with a clear high-performing region. Fig. 4 further shows a smooth accuracy landscape when varying the regularization weights α and β in Eq. (11), indicating that the topology reconstruction is not overly sensitive to precise regularization choices. We further evaluate DANCE across different GNN backbones in Appendix C.6.

Scaling Clients. We vary the number of participating sub-graph clients M to evaluate the stability of DANCE under different federation scales. Figure 4(right) shows that DANCE maintains competitive performance as the number of clients increases. This suggests DANCE can scale to larger federations without being overly sensitive.

Model-in-the-Loop Refresh Study. We additionally isolate the effect of *round-adaptive refresh* by comparing DANCE against (i) STATIC-DANCE, which performs condensation/evidence selection only once, (ii) REFRESH-CORE-ONLY, and (iii) REFRESH-TEXT-ONLY. This directly tests whether model-in-the-loop refresh is necessary under non-IID federation. Detailed experimental settings are provided in Appendix C.7.

6.5. Efficiency Analysis (Q4)

To answer Q4, we conduct both a theoretical and empirical efficiency analysis of DANCE, aiming to demonstrate its

Table 3. Theoretical time and space complexity comparison of different methods.

Method	Time Complexity	Space Complexity
FedAvg	$\mathcal{O}(Cn)$	$\mathcal{O}(n)$
FedC4	$\mathcal{O}(C(n \log n + Kq))$	$\mathcal{O}(n + Kq)$
FedTAD	$\mathcal{O}(Cn + n^2)$	$\mathcal{O}(n^2)$
LLM4RGNN	$\mathcal{O}(C(n^2 + n))$	$\mathcal{O}(n^2)$
LLaTA	$\mathcal{O}(Cn \log^2 n + (C^2 + 1)n)$	$\mathcal{O}(n + Cn)$
DANCE	$\mathcal{O}(C(n \log n + Kq))$	$\mathcal{O}(n + Kq)$

Table 4. End-to-end cost accounting on Cora and CiteSeer. DANCE counts the full local pipeline, including condensation refresh and LLM-related text processing.

Dataset	Method	Runtime	LLM calls	Total tokens
Cora	DANCE	0.299 h	956	0.26M
	LLaTA	2.492 h	4,062	3.66M
	LLM4RGNN	10.79 h	5,429	3.84M
CiteSeer	DANCE	0.433 h	1,316	0.43M
	LLaTA	2.972 h	4,779	4.30M
	LLM4RGNN	8.97 h	4,225	2.99M

computational and runtime advantages in federated text-attributed graph learning.

Theoretical Complexity Analysis. We analyze the theoretical *time and space* complexity of DANCE and compare it with representative federated and LLM-assisted baselines, including FedAvg, FedC4, FedTAD, LLM4RGNN, and LLaTA. Table 3 summarizes the corresponding costs. Let C denote the number of clients, n the number of local nodes, and K the number of condensed nodes per client. Formal derivations are deferred to Appendix E.3.

Empirical Runtime Evaluation. We evaluate the efficiency of DANCE by comparing training time against representative baselines. Beyond the per-node token saving reported in Table 1, we further report full end-to-end cost accounting in Table 4. This accounting includes condensation refresh, local LLM summary generation, chunk encoding, topology reconstruction, federated training, and evaluation under the same setting as the refresh study ($r = 0.08$, five clients, refresh every 10 rounds). Table 4 shows that DANCE remains substantially cheaper than LLM-heavy TAG-FGL baselines even when repeated condensation refresh and text processing are counted. We set the out-of-time (OOT) threshold to 72 hours for all methods; DANCE does not trigger OOT in our experiments, and its full runtime on the largest dataset, Children, is 8.98 hours. Figure 5 reports results on Cora and CiteSeer. LLM-based methods (e.g., LLaTA and LLM4RGNN) incur higher cost due to repeated LLM invocations, while FedTAD reduces overhead by offloading generation to the server but remains non-trivial. In contrast, DANCE achieves significantly lower training time under the same federated setting.

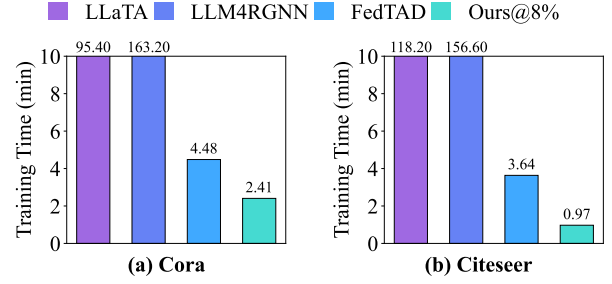


Figure 5. Training time comparison on Cora and CiteSeer. DANCE achieves substantially lower runtime than LLM-based methods and also yields noticeable reductions compared to conventional federated baselines.

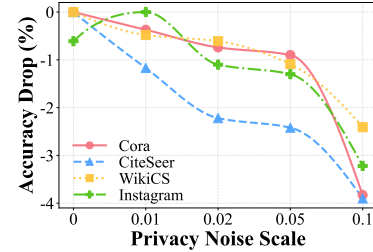


Figure 6. Utility under different representation-noise scales. DANCE retains competitive performance even under strong perturbation

6.6. Privacy Analysis (Q5)

To answer **Q5**, we apply secure aggregation (Bonawitz et al., 2017) to uploaded client updates and any server-side coordination signals. Since raw texts and decision traces are never transmitted, DANCE remains unchanged in how it generates local audit evidence.

Noise-Based Privacy Stress Test. We further conduct a preliminary noise-based privacy stress test by injecting Laplace noise into node representations used by the condensed training pipeline. Figure 6 reports the resulting utility under different noise scales. DANCE maintains competitive performance under moderate perturbations, suggesting that the proposed framework is robust to representation noise. This experiment should not be interpreted as a formal differential privacy guarantee; rigorous privacy accounting is left to future work. Implementation details are provided in Appendix C.9.

7. Conclusion

We presented **DANCE**, a round-adaptive condensation framework for federated learning on text-attributed graphs. By jointly budgeting node scale, neighbor usage, and chunk-level textual evidence, DANCE avoids repeatedly encoding and propagating redundant multi-hop texts, while refreshing retained evidence across rounds as the global model evolves under non-IID clients. DANCE communicates only standard model updates, remaining compatible with secure aggregation and optional noise-based perturbation,

yet provides locally available audit traces by materializing neighbor cues and supporting text chunks on-device. Empirically, DANCE achieves consistently stronger node classification performance and calibration than competitive TAG-FGL baselines under substantially reduced token processing and communication budgets. Future work includes more standardized interpretability and auditability evaluation, extending the framework to heterogeneous/temporal and multimodal graphs, and characterizing formal privacy-utility trade-offs under tighter accounting and stronger threat models.

Acknowledgement

This work was supported by New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0123403). This work was partially supported by the NSFC Grants U24A20255, U2241211, 62572055, 625B2021 and 62427808. This work was also supported by Guangdong Provincial Talent Program (No.2024TQ08X366).

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Baek, J., Jeong, W., Jin, J., Yoon, J., and Hwang, S. J. Personalized subgraph federated learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- Bonawitz, K., Ivanov, V., Kreuter, B., Marcedone, A., McMahan, H. B., Patel, S., Ramage, D., Segal, A., and Seth, K. Practical secure aggregation for privacy-preserving machine learning. In *CCS*, 2017.
- Chen, Z., Li, X., Zhu, Y., Li, R.-H., and Wang, G. Rethinking client-oriented federated graph learning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM '25*, pp. 393–402. ACM, November 2025. doi: 10.1145/3746252.3761228. URL <http://dx.doi.org/10.1145/3746252.3761228>.
- Chien, E., Peng, J., Li, P., and Milenkovic, O. Adaptive universal generalized pagerank graph neural network, 2021. URL <https://arxiv.org/abs/2006.07988>.
- Dai, Z., Shen, G., Hu, Y., Du, J., Han, X., and Kong, X. Fedgvd: Efficient federated graph learning via uni-directional distillation with dynamic virtual nodes. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM '25*, pp. 488–497, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400720406. doi: 10.1145/3746252.3761338. URL <https://doi.org/10.1145/3746252.3761338>.
- Fu, X., Zhang, B., Dong, Y., Chen, C., and Li, J. Federated graph machine learning: A survey of concepts, techniques, and applications. *ACM SIGKDD Explorations Newsletter*, 24(2):32–47, 2022.
- Hamilton, W. L., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- He, C., Balasubramanian, K., Ceyani, E., Yang, C., Xie, H., Sun, L., He, L., Yang, L., Philip, S. Y., Rong, Y., et al. Fedgraphnn: A federated learning benchmark system for graph neural networks. In *ICLR 2021 Workshop on Distributed and Private Machine Learning (DPML)*, 2021.
- Jin, W., Zhao, L., Zhang, S., Liu, Y., Tang, J., and Shah, N. Graph condensation for graph neural networks. *arXiv preprint arXiv:2110.07580*, 2021.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Li, X., Wu, Z., Zhang, W., Zhu, Y., Li, R.-H., and Wang, G. Fedgta: Topology-aware averaging for federated graph learning. *Proceedings of the VLDB Endowment*, 17(1): 41–50, 2023.
- Lopez-Ramos, L. M., Leiser, F., Rastogi, A., Hicks, S., Strümke, I., Madai, V. I., Budig, T., Sunyaev, A., and Hilbert, A. Interplay between federated learning and explainable artificial intelligence: a scoping review. *arXiv preprint arXiv:2411.05874*, 2024.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., and Aguera y Arcas, B. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.
- Pei, H., Wei, B., Chang, K. C.-C., Lei, Y., and Yang, B. Geom-gcn: Geometric graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2020.
- Qiu, Y. Default risk assessment of internet financial enterprises based on graph neural network. In *2023 IEEE 6th Information Technology, Networking, Electronic and Automation Control Conference*, volume 6, pp. 592–596. IEEE, 2023.

- Qu, Z., Yao, T., Liu, X., and Wang, G. A graph convolutional network based on univariate neurodegeneration biomarker for alzheimer’s disease diagnosis. *IEEE Journal of Translational Engineering in Health and Medicine*, 2023.
- Saifullah, S., Mercier, D., Lucieri, A., Dengel, A., and Ahmed, S. The privacy-explainability trade-off: unraveling the impacts of differential privacy and federated learning on attribution methods. *Frontiers in Artificial Intelligence*, 7:1236947, 2024.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Wu, S., Sun, F., Zhang, W., Xie, X., and Cui, B. Graph neural networks in recommender systems: a survey. *ACM Computing Surveys*, 55(5):1–37, 2022.
- Wu, Z., Li, X., Zhu, Y., Chen, Z., Yan, G., Yan, Y., Zhang, H., Ai, Y., Jin, X., Li, R.-H., et al. A comprehensive data-centric overview of federated graph learning. *arXiv preprint arXiv:2507.16541*, 2025a.
- Wu, Z., Zhu, Y., Li, X., Qiu, Z., Li, R.-H., Wang, G., and Zhou, C. Fedbook: A unified federated graph foundation codebook with intra-domain and inter-domain knowledge modeling. *arXiv preprint arXiv:2510.07755*, 2025b.
- Yan, B., Zhang, Z., Sun, H., Zhang, M., Cao, Y., and Shi, C. Data-centric federated graph learning with large language models. *arXiv preprint arXiv:2503.19455*, 2025.
- Zhang, H., Li, X., Zhu, Y., and Hu, L. Rethinking federated graph learning: A data condensation perspective, 2025a. URL <https://arxiv.org/abs/2505.02573>.
- Zhang, K., Yang, C., Li, X., Sun, L., and Yiu, S. M. Sub-graph federated learning with missing neighbor generation. *Advances in neural information processing systems*, 34:6671–6682, 2021.
- Zhang, Z., Li, X., Zhu, L., Zeng, G., Fan, B., Wen, Y., Qin, H., Li, R.-H., and Wang, G. Unlocking graph structure learning with tree-guided large language models. *arXiv preprint arXiv:2503.21223*, 2025b.
- Zhang, Z., Wang, X., Zhou, H., Yu, Y., Zhang, M., Yang, C., and Shi, C. Can large language models improve the adversarial robustness of graph neural networks? In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pp. 2008–2019, 2025c.
- Zheng, X., Zhang, M., Chen, C., Nguyen, Q. V. H., Zhu, X., and Pan, S. Structure-free graph condensation: From large-scale graphs to condensed graph-free data. *Advances in Neural Information Processing Systems*, 36: 6026–6047, 2023.
- Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., and Koutra, D. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in Neural Information Processing Systems*, volume 33, pp. 7793–7804, 2020.
- Zhu, Y., Li, X., Jia, J., Hu, M., Wu, D., and Qiu, M. Towards effective federated graph foundation model via mitigating knowledge entanglement. *arXiv preprint arXiv:2505.12684*, 2025.

A. Code Availability

For reproducibility, we provide the official implementation of **DANCE** at <https://anonymous.4open.science/r/test-repo-9386/>.

B. Methodology in Details

Difficulty scores for budgeted evidence. To prioritize informative (hard) evidence under tight budgets, we define a model-conditioned difficulty score for each node based on predictive uncertainty from the last-round global model:

$$u_v \triangleq H\left(p^{(t-1)}(y | v)\right) = - \sum_{c=1}^C p^{(t-1)}(c | v) \log p^{(t-1)}(c | v), \quad (21)$$

where larger u_v indicates higher uncertainty (harder nodes). We use $\{u_v\}$ only for evidence prioritization (e.g., selecting 2-hop candidates) and do not treat it as a pseudo-label signal.

C. Experiment in Details

C.1. Computation Resource

Experiments are conducted on a Linux server equipped with 2 Intel(R) Xeon(R) Gold 6240 CPUs @ 2.60GHz (36 cores per socket, 72 threads total), 251GB RAM (with approximately 216GB available), and 4 NVIDIA A100 GPUs with 40GB memory each. The software environment includes Python 3.10.19 and PyTorch 2.4.1 with CUDA 12.4. We adopt a multi-GPU training setup where each simulated client is assigned to a dedicated GPU during federated optimization.

C.2. Overall Performance: Experimental Settings and Protocol

This subsection details the experimental settings and evaluation protocol used for the overall performance comparison in Section 6.2.

Unified baseline adaptation. For graph condensation methods originally proposed in centralized settings, we apply condensation *locally* on each client’s subgraph under the federated setting. The condensed representations are then used within the same federated training and aggregation pipeline as other methods. For baselines not specifically designed for text-attributed graphs, we adopt the same SBERT encoder as DANCE to obtain node text representations, ensuring a consistent text processing backbone across methods.

This unified treatment guarantees that observed performance differences stem from the learning, condensation, and aggregation strategies, rather than discrepancies in textual feature extraction or encoder capacity.

Evaluation metrics. We report **Accuracy** as the primary predictive metric, following standard practice in semi-supervised node classification on citation and text-rich graphs. Additional metrics such as macro-F1 and heterogeneity-aware scores are reported where appropriate to capture performance under non-IID federated settings.

Federated training protocol. Unless otherwise stated, all methods are evaluated under the same federated schedule, including identical client partitions, communication rounds, and local training epochs. Hyper-parameters for all methods are selected on a validation split using a matched tuning budget to avoid unfair optimization advantages.

Randomness and reporting. All reported results are averaged over five independent random seeds and presented as mean $\pm$ standard deviation. For convergence analysis, we track test accuracy across communication rounds using the same evaluation checkpoints for all methods.

Overall, this protocol ensures a fair and controlled comparison across heterogeneous baselines, isolating the impact of condensation, neighbor-gated aggregation, and evidence selection mechanisms introduced by DANCE.

C.3. Fine-Grained Architectural Ablations

To further isolate the effect of each architectural choice, we conduct fine-grained ablations on Cora and CiteSeer under the same setting used in the rebuttal study: condensation ratio $r = 0.08$, five clients, refresh every 10 communication rounds, and three random seeds. These experiments use the chunks-only setting so that the ablation focuses on the condensation and reconstruction modules rather than on variation from local summary generation.

Table 5. Fine-grained ablations on Cora and CiteSeer under the same setting. For each module, we compare the full DANCE design with three alternative strategies while keeping the other modules unchanged.

Module	Method	Cora (0.08)		CiteSeer (0.08)	
		Overall-F1	Acc	Overall-F1	Acc
Full	DANCE	88.36 ± 0.52	88.87 ± 0.43	77.16 ± 0.83	80.19 ± 0.76
TGR	Cosine similarity	84.39 ± 0.76	85.88 ± 0.64	73.92 ± 0.63	77.49 ± 0.16
	Euclidean distance	85.30 ± 1.27	86.74 ± 0.93	72.70 ± 0.71	76.48 ± 0.79
	w/o TGR	86.32 ± 0.64	87.08 ± 0.52	73.32 ± 0.88	77.32 ± 0.82
TC	Softmax gate	86.12 ± 0.25	87.68 ± 0.22	74.90 ± 1.25	76.97 ± 0.49
	Uniform gate	84.42 ± 0.15	86.06 ± 0.20	73.97 ± 0.80	77.60 ± 0.47
	w/o TC	82.92 ± 0.71	85.14 ± 0.61	73.07 ± 1.02	77.11 ± 0.94
NC	K-means clustering	84.77 ± 0.58	86.24 ± 0.29	71.65 ± 0.73	76.10 ± 0.79
	Degree-based selection	80.68 ± 0.18	81.93 ± 0.34	69.25 ± 0.93	73.48 ± 0.80
	w/o NC	81.04 ± 0.68	81.22 ± 0.56	65.44 ± 0.95	75.19 ± 0.89

These controlled comparisons show that each architectural component contributes to the final performance. Across both datasets and both metrics, the full DANCE design consistently improves over replacing topology reconstruction, text-condensation gating, or node condensation with simpler alternatives under the same resource budget.

C.4. Structured Prompt for Evidence-Constrained Summary Generation

To generate reusable summaries after chunk selection, we use a lightweight LLM with a structured, evidence-constrained prompt. The prompt explicitly separates core-node evidence from neighbor evidence and further groups neighbor chunks by hop and category. This design enforces that the LLM only compresses the provided evidence into a concise summary, without introducing external knowledge or cross-group inference.

After chunk-level evidence selection, DANCE generates a reusable textual summary using a lightweight large language model (LLM) with a strictly structured, evidence-constrained prompt. In our implementation, we use *DeepSeek* as the LLM backbone due to its strong instruction-following ability and favorable efficiency–quality trade-off, which makes it suitable for large-scale, repeated summary generation.

The prompt is explicitly designed to restrict the role of the LLM to *faithful compression* rather than open-ended reasoning. Specifically, the prompt separates evidence into core-node chunks and neighbor chunks, and further organizes neighbor evidence by hop distance and category. This structure mirrors the hierarchical evidence selection procedure in DANCE and ensures that the LLM is aware of the structural origin of each chunk without being asked to infer or reason across groups.

To prevent hallucination and unintended information fusion, the prompt enforces several hard constraints: (i) the LLM can only use the provided evidence chunks; (ii) no external knowledge, background information, or unstated assumptions are allowed; and (iii) cross-group inference between different neighbors or hops is explicitly prohibited. As a result, the generated summary preserves the factual content of the selected evidence while remaining compact and human-readable.

The generated summaries are short (2–4 sentences), deterministic under low-temperature decoding, and can be cached and reused across communication rounds. Importantly, since the summary generation is applied only to the already condensed evidence set, the additional computational overhead introduced by the LLM remains modest. This design allows DANCE to benefit from semantic compression via LLMs without sacrificing efficiency or faithfulness.

Figure 7 shows the structured prompt template used in our experiments.

Structured Prompt for Evidence-Constrained Summary Generation**[System]**

Scientific summarization assistant. Produce a concise and faithful summary strictly based on the provided evidence.

[Task]

Summarize the core idea expressed in the evidence chunks for downstream node classification.

[Constraints]

- Use only the provided evidence chunks.
- Do not add external knowledge or background.
- Do not infer relationships across different neighbor groups.
- Preserve the factual content of each group.
- Use concise, declarative academic language.
- Length: 2–4 sentences.

[Evidence Chunks]**(A) Core node evidence:**

<chunks from the core node>

(B) Neighbor evidence grouped by hop and category:

<1-hop neighbors, category 1: chunks>
 <1-hop neighbors, category 2: chunks>
 <2-hop neighbors, category 1: chunks>
 <2-hop neighbors, category 2: chunks>

[Output]

Summary:

Figure 7. Structured prompt template with hop- and category-aware evidence grouping. The LLM receives pre-grouped evidence chunks and generates summaries without cross-group inference.

C.5. Case Study: Faithful Evidence Selection on Cora

Setup. We present an illustrative qualitative example on the Cora citation network to examine evidence selection at the chunk level. No manual filtering or post hoc modification is applied. The same automated pipeline as in quantitative experiments is used, while summary-level sufficiency and comprehensiveness are evaluated separately in subsequent experiments.

Evidence selection. Evidence selection is performed automatically by DANCE via neighbor-gated aggregation and chunk-level selection. Each chunk is embedded and scored by its contribution to the node representation under neighbor-aware message aggregation, and the final evidence set is formed by selecting the top-ranked chunks under the same budget constraints used at test time. Based on the selected chunks, we further use a lightweight LLM to generate a short, reusable summary under an evidence-constrained prompt, which compresses the evidence while preserving its factual content.

In this example, DANCE selects human-readable chunks from the core node that are highly relevant to the *Neural Networks* label. It further retrieves evidence from hop neighbors that does not explicitly mention “neural networks” but remains strongly aligned with the target concept (e.g., stability and effectiveness in changing environments). Meanwhile, loosely related content such as references to dynamic programming is not selected, reflecting DANCE’s preference for evidence that is directly relevant to the prediction task. The LLM-generated summary faithfully consolidates the selected evidence into a compact description, without introducing information beyond the retrieved chunks.

Faithfulness evaluation protocol. We evaluate faithfulness using paired *sufficiency* and *comprehensiveness* tests, following standard practice. For sufficiency, we retain only the DANCE-selected chunks and mask all remaining text. For comprehensiveness, we remove the selected evidence while keeping all other text intact. Random chunk baselines of equal length are used as controls. As shown in Figure 9, retaining only the selected chunks largely preserves predictive performance, whereas removing them results in a noticeably larger degradation than random deletion, indicating that the selected evidence is both sufficient and necessary for the model’s prediction.

Original Paper (Excerpt)	Selected Evidence Chunks by DANCE
Title: Exploration and Model Building in Mobile Robot Domains Label: Neural Networks Abstract (excerpt): <i>Real-world experiences are generalized via two artificial neural networks that encode the characteristics of the robot’s sensors, as well as the characteristics of typical environments the robot is assumed to face.</i> <i>Once trained, these networks allow for knowledge transfer across different environments ...</i> <i>... an efficient dynamic programming method is employed in background ...</i>	Evidence selected in core node: 1. “Real-world experiences are generalized via two artificial <u>neural networks</u> that encode the characteristics of the robot’s sensors.” 2. ... Evidence selected in hop neighbors: 1. “The resulting learning system is more stable and effective in changing environments than plain <u>backpropagation</u>...” 2. ...
LLM-generated summary: Real-world experiences are modeled using two artificial neural networks that encode sensor characteristics. Once trained, the networks enable knowledge transfer across environments and yield a learning system that is more stable and effective in changing conditions than plain backpropagation.	

Figure 8. **Illustrative example on the Cora dataset (Neural Networks class).** The original abstract contains extensive descriptions related to robotics, neural networks, and system-level details. DANCE selectively extracts evidence that is directly relevant to the node prediction. The selected evidence is presented as intact, human-readable text, and a lightweight LLM generates a concise summary using only the selected chunks.

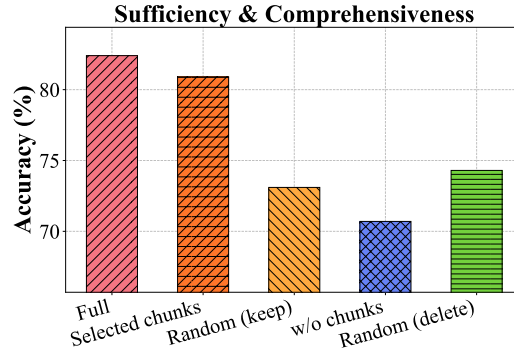


Figure 9. Chunk-level faithfulness analysis on Cora. The plot compares model accuracy under different evidence conditions, including the full text, retaining only DANCE-selected chunks (sufficiency), removing the selected chunks (comprehensiveness), and corresponding random controls with the same budget.

Takeaway. This case study demonstrates that DANCE selects evidence that is both sufficient and necessary for prediction under the same automated pipeline used in evaluation. The selected evidence remains compact, human-interpretable, and faithful to the model’s actual decision process, rather than reflecting post hoc or naively keyword-driven explanations.

C.6. Robustness to Different GNN Backbones

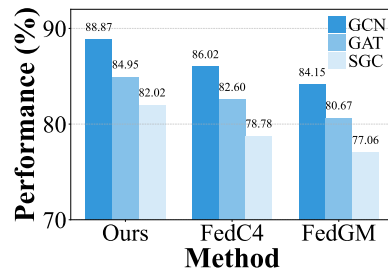


Figure 10. Performance comparison under different GNN backbones (GCN, GAT, and SGC) on Cora with 8% labeled data. All methods use the same hyper-parameter settings as in the main experiments.

We evaluate the robustness of DANCE to different GNN backbones on the Cora dataset under the 8% label setting. All hyper-parameters and training protocols are kept identical to those used in the main experiments, and only the graph encoder backbone is varied. Specifically, we instantiate the backbone with GCN, GAT, and SGC, respectively.

Figure 10 reports the corresponding results. Across different backbones, DANCE consistently outperforms the compared compression-based methods, while the performance variation induced by changing the backbone is of similar magnitude to that observed for FedC4 and FedGM. This suggests that DANCE does not exhibit abnormal sensitivity to the choice of GNN architecture, and its behavior remains stable across common backbone designs.

C.7. Model-in-the-Loop Refresh Study

Experimental Setup. We investigate the effect of round-adaptive model-in-the-loop refresh under the standard subgraph federated learning setting. All variants share the same federated schedule, client partition strategy, communication rounds, and training hyper-parameters. Unless otherwise stated, we follow the default configuration used in Section 6, including the same condensation ratio, neighbor budgets, and chunk budgets. The only difference across variants lies in whether and how the condensed node core and the selected textual evidence are refreshed as the global model evolves across rounds.

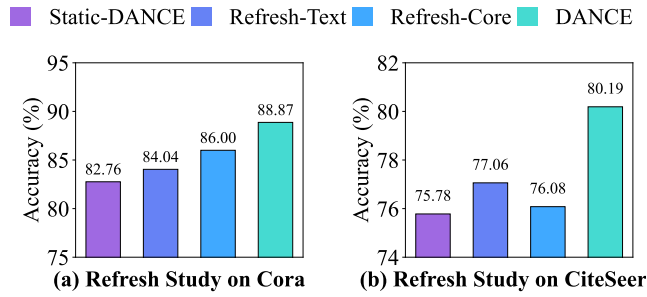


Figure 11. Model-in-the-loop refresh study on Cora and CiteSeer. Full DANCE refreshes both the condensed node core and textual evidence every 10 communication rounds, while other variants refresh only partial components or remain static.

Compared Variants. We consider four variants to disentangle the contribution of structural and textual refresh under model-in-the-loop training. All variants share the same backbone architecture, condensation ratio, budget settings, and federated training schedule, and differ only in whether the condensed node core and the selected textual evidence are updated as the global model evolves. **DANCE** is the full method, which refreshes both the condensed node core and the selected evidence chunks every 10 communication rounds using the current global model. **Static-DANCE** performs condensation and evidence selection only once at $t=1$ and reuses the resulting condensed graph and evidence for all subsequent rounds, thereby removing model adaptivity entirely. **Refresh-Core-Only** refreshes the condensed node core every 10 communication rounds while keeping textual evidence (neighbor selection and chunk selection) fixed after initialization, isolating the effect of structural refresh. **Refresh-Text-Only** refreshes textual evidence every 10 communication rounds while the condensed node core remains fixed throughout training, isolating the effect of evidence refresh.

Datasets and Metrics. We report node classification accuracy (%) on **Cora** and **CiteSeer**. We use the same non-IID client partition protocol, communication schedule, and hyper-parameters as in Section 6, with identical train/validation/test splits on the global test set. Results are averaged over five random seeds.

Results and Analysis. Figure 11 summarizes the results. On both datasets, full DANCE consistently outperforms Static-DANCE, with an absolute improvement of +6.1% on Cora and +4.4% on CiteSeer. This gap indicates that performing condensation and evidence selection only once is insufficient, as the relevance of both structural cores and textual evidence changes as the global model is updated across federated rounds.

The partial-refresh variants recover part of the lost performance, suggesting that refreshing either the structural or textual component alone is beneficial. However, both **Refresh-Core-Only** and **Refresh-Text-Only** remain consistently inferior to full DANCE, highlighting that structural condensation and evidence selection are tightly coupled. Refreshing only one component leads to a mismatch between the evolving representation space and the fixed component, which limits performance under non-IID federated training.

Overall, these results confirm that jointly refreshing both the condensed node core and the selected textual evidence in a model-in-the-loop manner is necessary to fully exploit the benefits of DANCE in federated settings.

C.8. Robustness to Low-Quality Text

We further test whether topology reconstruction collapses when text attributes are incomplete or misleading. We evaluate two raw-text corruptions before running the full pipeline: (i) whitespace masking, which removes information by randomly replacing words with blanks, and (ii) text swapping, which injects misinformation by replacing a fraction of node texts with texts from other nodes. All runs use the 8% condensation ratio, and the numbers below report accuracy drops relative to the uncorrupted setting.

Table 6. Accuracy drops under low-quality text, measured relative to the uncorrupted setting.

Corruption	Ratio	Cora drop	CiteSeer drop	WikiCS drop
Whitespace masking	0.2	−2.69	−2.59	−3.10
	0.4	−4.14	−3.32	−6.03
	0.5	−4.90	−4.45	−7.20
Text swapping	0.10	−3.70	−3.50	−2.18
	0.15	−4.49	−5.17	−2.51
	0.20	−5.51	−6.68	−3.31

The masking and swapping results show progressive degradation rather than abrupt failure. This indicates that DANCE benefits from textual evidence, but the fused graph–text design and sparse topology reconstruction do not depend on perfectly clean text attributes.

C.9. Noise Injection Implementation Details

We describe the concrete implementation of the noise-based perturbation experiment reported in Section 6.6.

Noise injection mechanism. At each communication round, we perturb the condensed node representations on the client side before forming the local model update. Specifically, we add element-wise Laplace noise to the condensed feature matrix:

$$\tilde{\mathbf{X}} = \mathbf{X} + \text{Laplace}(0, b),$$

where the noise scale b controls the perturbation strength.

Normalization and stability. After noise injection, the perturbed representations are ℓ_2 -normalized to prevent scale explosion and to stabilize subsequent aggregation and training. No noise is added to raw texts, intermediate decision traces, or local evidence packs, which are never transmitted off-device.

Scope and limitation. The applied perturbation is an empirical stress test for robustness to noisy condensed representations, not a standalone formal differential privacy mechanism. We do not claim a formal privacy guarantee without sensitivity calibration, composition accounting, and a threat-model-specific privacy proof. Local audit artifacts, including neighbor importance scores and selected evidence chunks, remain entirely local and are unaffected by the perturbation experiment.

C.10. Datasets & Simulation Method

We evaluate **DANCE** on eight widely adopted text-attributed graph (TAG) datasets spanning multiple domains, including citation networks, one knowledge network, two social networks, and two e-commerce networks. Among these datasets, the e-commerce datasets exhibit stronger heterophily, while the remaining datasets are relatively homophilic. The details of these TAG datasets are summarized in Table 7. The description of the datasets for each domain is as follows:

Citation Networks. Cora, Citeseer, and Arxiv2023 are benchmark citation network datasets. Nodes represent academic papers, and edges denote citation relationships. Node texts consist of paper titles and abstracts, and labels correspond to research areas or subject categories.

Knowledge Network. WikiCS is a benchmark dataset of knowledge networks constructed from Wikipedia articles in computer science. Nodes represent articles, and edges indicate hyperlink relationships between articles. Node texts are

obtained from article titles and abstracts, and labels denote different branches of computer science.

Social Networks. Instagram and Reddit are benchmark datasets of social networks. Nodes represent users, and edges represent social relationships. Textual features are derived from user profiles or recent posts, and labels correspond to different types of users or communities.

E-commerce Networks. Ratings and Children are benchmark datasets of e-commerce networks. Nodes represent products, and edges indicate relations such as co-purchase or co-view. Textual attributes include product names and descriptions, while labels correspond to product categories or rating groups.

Table 7. Statistics of Datasets.

Dataset	Nodes	Edges	Classes	Train/Val/Test	Text Information	Domain
Cora	2,708	10,556	7	60/20/20	Title and Abstract of Paper	Citation
Citeseer	3,186	8,450	6	60/20/20	Title and Abstract of Paper	Citation
WikiCS	11,701	431,726	10	5/22.5/50	Title and Abstract of Article	Knowledge
Arxiv2023	46,198	38,863	38	63/20/17	Title and Abstract of Paper	Citation
Instagram	11,339	144,010	2	10/10/90	Personal Profile of User	Social
Reddit	33,434	269,442	2	10/10/90	Last 3 Posts of User	Social
Ratings	24,492	186,100	5	25/25/50	Name of Product	E-commerce
Children	76,875	1,162,522	24	60/20/20	Name and Description of Book	E-commerce

C.11. Additional Heterophilic Evaluation

To further address robustness under heterophily, we add supplementary experiments on standard heterophilic benchmarks and more recent heterophilic stress tests. The WebKB datasets (Wisconsin, Cornell, and Texas) are standard but limited proxies; recent benchmark efforts such as GraphLand and GraphBench further show that realistic heterophily can differ from these small datasets. We therefore report the WebKB results separately from Roman Empire and Questions, and treat all of them as supplementary stress tests rather than replacements for the main TAG benchmark suite.

Table 8. Additional heterophilic evaluation on WebKB and recent stress-test benchmarks. Results are accuracy (%).

Dataset	Ratio	FedAvg	FedSage+	FedGTA	GCond	FedC4	FedGVD	Ours
Wisconsin	0.4	77.6 $\pm$ 2.7	79.0 $\pm$ 3.3	77.8 $\pm$ 4.1	72.8 $\pm$ 2.7	75.4 $\pm$ 2.4	74.1 $\pm$ 2.7	77.8 $\pm$ 2.5
	0.6				77.6 $\pm$ 1.5	80.1 $\pm$ 1.2	79.3 $\pm$ 1.5	82.0 $\pm$ 2.1
Cornell	0.4	72.3 $\pm$ 4.3	73.1 $\pm$ 3.3	72.6 $\pm$ 4.2	63.9 $\pm$ 0.8	65.5 $\pm$ 0.3	64.9 $\pm$ 0.4	68.1 $\pm$ 0.7
	0.6				67.3 $\pm$ 0.5	69.4 $\pm$ 0.2	68.7 $\pm$ 0.2	70.8 $\pm$ 0.4
Texas	0.4	72.8 $\pm$ 2.2	73.7 $\pm$ 4.0	79.2 $\pm$ 0.3	67.8 $\pm$ 1.5	69.2 $\pm$ 1.4	68.4 $\pm$ 1.6	71.7 $\pm$ 2.4
	0.6				75.9 $\pm$ 1.8	77.5 $\pm$ 1.2	76.8 $\pm$ 1.8	79.5 $\pm$ 1.2
Roman Empire	2%	81.6 $\pm$ 0.8	84.4 $\pm$ 0.7	87.2 $\pm$ 0.6	75.4 $\pm$ 1.1	78.8 $\pm$ 0.9	77.9 $\pm$ 1.0	86.5 $\pm$ 0.8
	4%				81.1 $\pm$ 0.9	84.0 $\pm$ 0.7	83.2 $\pm$ 0.8	88.1 $\pm$ 0.6
Questions	2%	76.2 $\pm$ 0.9	78.4 $\pm$ 0.8	79.8 $\pm$ 0.7	72.8 $\pm$ 1.0	75.1 $\pm$ 0.8	74.4 $\pm$ 0.9	79.3 $\pm$ 0.7
	4%				76.6 $\pm$ 0.8	78.5 $\pm$ 0.6	77.9 $\pm$ 0.7	81.0 $\pm$ 0.5

These results indicate that DANCE remains competitive beyond the two heterophilic TAG datasets used in the main benchmark. At the same time, the gap between WebKB-style benchmarks and newer heterophily benchmarks suggests that building standardized text-attributed heterophily benchmarks is an important direction for future evaluation.

C.12. Baselines and Experimental Settings

C.12.1. BASELINES.

We compare DANCE with representative baselines spanning Classical FL and FGL, Graph condensation, Federated graph condensation, and LLM-based TAG-FGL methods, in order to comprehensively evaluate its effectiveness.

Classical FL and FGL methods. **FedAvg** (McMahan et al., 2017) is a classical federated learning algorithm that aggregates client models by weighted averaging. We include it as a standard optimization-based baseline to assess whether performance gains stem from data modeling rather than federated optimization alone. **FedSage+** (Zhang et al., 2021) is a subgraph-based federated graph learning method that extends FedSage by explicitly modeling missing neighbors across local subgraphs. It introduces a neighbor generation mechanism to mitigate structural incompleteness caused by subgraph partitioning, and serves as a canonical structure-aware baseline in subgraph-FL settings. We include it as a canonical structure-aware baseline in subgraph-FL settings. **FedGTA** (Li et al., 2023) is a topology-aware federated graph learning method that introduces topology-aware aggregation to perform personalized model updates across clients. By incorporating local graph structure into the aggregation process through topology-aware smoothing confidence and mixed neighbor features, FedGTA effectively mitigates structural heterogeneity among client subgraphs. We include it to compare against recent advances in structure-aware subgraph-FL.

Graph condensation methods. **GCond** (Jin et al., 2021) is a centralized graph condensation method that learns a compact synthetic graph whose training dynamics approximate those of the original graph. It jointly optimizes synthetic node features and graph structure via gradient matching. We include it as a representative condensation baseline to isolate the effectiveness of condensation mechanisms. **SFGC** (Zheng et al., 2023) is a centralized structure-free graph condensation method that compresses graphs into a compact set of synthetic nodes without explicitly constructing a condensed graph topology. It incorporates structural information implicitly during condensation while avoiding storing or optimizing condensed edges, and is included to contrast with DANCE in terms of whether explicitly modeling condensed structure benefits downstream performance.

Federated graph condensation methods. **FedC4** (Chen et al., 2025) is a federated graph condensation framework that compresses local graphs before communication. We include it as a representative federated condensation baseline to contrast with DANCE in terms of data abstraction and structural modeling. **FedGVD** (Dai et al., 2025) is a generative federated condensation approach that employs server-side virtual nodes and knowledge distillation to share global information. We include it to compare with data-centric federated condensation methods that do not preserve explicit graph structure. **FedGM** (Zhang et al., 2025a) is a data condensation-based federated graph learning framework that addresses data heterogeneity by aggregating condensed graphs as optimization carriers. We include it as a representative baseline for evaluating how different condensation strategies handle heterogeneity in federated graph learning.

LLM-based TAG-FGL methods. We additionally compare against recent TAG/FGL methods that explicitly incorporate large language models (LLMs) as a module for structure- or data-level enhancement, rather than performing condensation or budgeted evidence selection. **LLM4RGNN** (Zhang et al., 2025c) is an LLM-assisted robust graph learning framework that uses LLMs to infer, validate, and repair graph relations under topology perturbations (e.g., missing, noisy, or adversarial edges), and then distills the resulting structural signals into lightweight local models so that inference can be performed without changing the backbone GNN. We include LLM4RGNN to represent LLM-based approaches where language models serve as a *structure-reasoning oracle*, improving robustness by correcting graph connectivity instead of selecting a compact evidence set. **LLaTA** (Zhang et al., 2025b) integrates LLMs into graph structure learning by reformulating topology refinement as a language-aware tree optimization problem, where the LLM provides in-context judgments to guide tree-based updates to the graph structure without additional parameter tuning. We include LLaTA as a representative of *prompt-driven* structure refinement for text-attributed graphs, which primarily strengthens topology modeling rather than reducing computation via condensation. **LLM4FGL** (Yan et al., 2025) is an LLM-driven federated graph learning framework that addresses cross-client heterogeneity by augmenting local data through language-model-based neighbor generation and federated reflection, decomposing the LLM usage into data generation and relation inference so that heterogeneity can be mitigated through data-level enrichment without modifying the local GNN architecture. We include LLM4FGL as a representative LLM-based baseline that tackles heterogeneity from a *data-centric* perspective, providing a direct comparison to DANCE in terms of how LLMs are used (augmentation/reflection versus budgeted evidence selection and condensation).

Table 9. Hyper-parameter settings for DANCE.

Hyper-parameter	Value / Search Range
Backbone GNN	2-layer GCN
Hidden dimension	{64, 128}
Number of clients	5
Communication rounds	200
Local epochs per round	3
1-hop budget	3
2-hop budget	2
Summary mixing ratio (<i>mix</i>)	{0.4, 0.6, 0.8}
Topology parameter α	{4, 8, 12}
Topology parameter β	{3, 5, 10}
GNN learning rate	1e−2
GNN Weight decay	5e−4
GNN Dropout rate	0.5

C.12.2. HYPER-PARAMETERS.

Table 9 summarizes the hyper-parameter settings used in all experiments. For both clients and the central server, we adopt a 2-layer GCN as the backbone model, with the hidden dimension selected from {64, 128}. Unless otherwise specified, the number of participating clients is fixed to 5. Federated training is conducted for 200 communication rounds, and in each round, every client performs 3 local training epochs before model aggregation.

For the proposed DANCE framework, we employ a hop-wise neighbor-gated condensation strategy. The condensation budgets for 1-hop and 2-hop neighbors are set to 3 and 2, respectively. The summary mixing ratio *mix* is selected from {0.4, 0.6, 0.8}. For the topology reconstruction module, the hyper-parameters α and β are searched within {4, 8, 12} and {3, 5, 10}, respectively. Across all GNN backbones, the learning rate is set to 1e−2, the weight decay is set to 5e−4, and the dropout rate is fixed to 0.5. All hyper-parameters are selected based on validation performance. For each experiment, we report the mean and standard deviation over 5 seeds. For textual encoding, we use a frozen pretrained SBERT encoder (all-MiniLM-L6-v2) to extract node-level text representations across all datasets.

D. Algorithm in Pseudo Code

Algorithm 1 DANCE: Round-wise Condensation and Federated Training

Require: Rounds T ; client fraction ρ ; refresh interval $R=10$; budgets $K, (B_0, B_1, B_2), B_{\text{tok}}, k$.

```

1: Initialize global parameters  $\omega^{(0)} = (\theta^{(0)}, \phi^{(0)})$ .
2: for  $t = 1, \dots, T$  do
3:   Server samples clients  $\mathcal{M}_t$  (rate  $\rho$ ) and broadcasts  $\omega^{(t-1)}$ .
4:   for all  $m \in \mathcal{M}_t$  in parallel do
5:     Text caching: chunk texts into  $e_{u,r}$  and compute embeddings  $t_u$  (refresh if encoder changes).
6:     if  $t = 1$  or  $t \bmod R = 0$  then
7:       Node condensation: compute  $(\tilde{y}_v, \pi_v)$  (Eq. (3)) and  $u_v$  (Eq. (21)); cluster per-class embeddings to obtain  $\{\mathcal{P}_c\}$  and select  $\hat{V}_t^{(m)}$  under label-proportional budgets.
8:       Text condensation: build multi-hop candidates  $\tilde{\mathcal{N}}_v$  and pre-filtering 2-hop with  $u_v$ ; compute neighbor gates  $\alpha$  (Eq. (5)) and context  $c_v$  (Eq. (6)); distill  $\tilde{t}_v$  (Eq. (8)) and store evidence packs/neighbor cues locally.
9:       Topology reconstruction: fuse  $X$  (Eq. (9)); update  $Z$  (Eq. (11)); form  $\hat{A}_t^{(m)}$  (Eq. (12)).
10:    else
11:      Reuse cached  $\hat{V}_t^{(m)}, \tilde{t}_v$ , evidence packs, and  $\hat{A}_t^{(m)}$  from the latest refresh round.
12:    end if
13:    Local training & upload: train on the condensed TAG using standard subgraph-FL, obtain update  $\Delta\omega_m^{(t)}$ , and upload  $\Delta\omega_m^{(t)}$  only.
14:    Local audit traces: summarize evidence packs into keywords/evidence sentences using an on-device LLM; keep all traces local.
15:  end for
16:  Server aggregates  $\{\Delta\omega_m^{(t)}\}$  to obtain  $\omega^{(t)}$ .
17: end for
18: return  $\omega^{(T)}$ .
    
```

Algorithm 2 Label-aware Node Condensation (per client m at round t)

Input: Global model $\omega^{(t-1)}$; local TAG $\mathcal{G}_m = (V^{(m)}, E^{(m)}, S^{(m)})$; labeled set $V_L^{(m)}$ with $\{y_v\}$; threshold τ ; prototype cap P ; node budget K (or ratio r)

Output: Condensed core $\hat{V}_t^{(m)}$; pseudo labels/confidence $\{(\tilde{y}_v, \pi_v)\}$

```

1 Forward (once): run  $\omega^{(t-1)}$  on  $\mathcal{G}_m$  Obtain embeddings  $\{z_v\}_{v \in V^{(m)}}$  Obtain predictive distributions  $p^{(t-1)}(y | v)$ 
2 foreach  $v \in V^{(m)}$  do
3   if  $v \in V_L^{(m)}$  then
4      $\tilde{y}_v \leftarrow y_v$ 
5   else
6      $\tilde{y}_v \leftarrow \arg \max_c p^{(t-1)}(c | v)$ 
7    $\pi_v \leftarrow \max_c p^{(t-1)}(c | v)$ 
8 foreach class  $c$  do
9    $V_c^{(m)} \leftarrow \{v \in V^{(m)} : \tilde{y}_v = c, \pi_v \geq \tau\}$ 
10 foreach class  $c$  with  $V_c^{(m)} \neq \emptyset$  do
11    $P_c \leftarrow \min(P, |V_c^{(m)}|)$   $\mathcal{P}_c \leftarrow \text{Cluster}(\{z_v\}_{v \in V_c^{(m)}}; P_c)$ 
12 foreach  $v$  with  $\pi_v \geq \tau$  do
13    $s_v \leftarrow \max_{p \in \mathcal{P}_{\tilde{y}_v}} \kappa(z_v, p)$ 
14 if budget is given as ratio then
15    $K \leftarrow \lceil r \cdot |V^{(m)}| \rceil$ 
16  $N_{\text{conf}} \leftarrow \sum_c |V_c^{(m)}|$  foreach class  $c$  do
17    $\bar{K}_c \leftarrow K \cdot \frac{|V_c^{(m)}|}{N_{\text{conf}}}$   $K_c \leftarrow \lfloor \bar{K}_c \rfloor$   $\delta_c \leftarrow \bar{K}_c - K_c$ 
18  $R \leftarrow K - \sum_c K_c$  Sort classes by  $\delta_c$  in descending order for  $j = 1$  to  $R$  do
19   Let  $c_j$  be the  $j$ -th class in the sorted order  $K_{c_j} \leftarrow K_{c_j} + 1$ 
20  $\hat{V}_t^{(m)} \leftarrow \emptyset$  foreach class  $c$  do
21    $\hat{V}_t^{(m)} \leftarrow \hat{V}_t^{(m)} \cup \text{TopK}_{K_c}(\{(v, s_v) : v \in V_c^{(m)}\})$ 
22 Return  $\hat{V}_t^{(m)}, \{(\tilde{y}_v, \pi_v)\}$ 
    
```

Algorithm 3 Hierarchical Text Condensation (per client m at round t)

Input: Frozen encoder Enc ; chunker $\mathcal{S}(\cdot)$; pooling Pool ; (optional) cache $\{e_{u,r}, t_u\}$; core $\hat{V}_t^{(m)}$; last-round graph encoder in $\omega^{(t-1)}$; difficulty $\{u_w\}$; hop budgets (B_0, B_1, B_2) ; token budget B_{tok} ; $\Pi_B(\cdot)$; entmax; parameters (W_q, W_k, W_s, γ)

Output: Evidence embeddings $\{\tilde{t}_v\}$; neighbor gates $\{\alpha^{(\ell)}\}$; evidence packs $\{\pi\}$ (kept local)

```

23 if cache is missing then
24   foreach  $u \in V^{(m)}$  do
25      $\{s_{u,r}\}_{r=1}^{R_u} \leftarrow \mathcal{S}(s_u)$  foreach  $r \in \{1, \dots, R_u\}$  do
26        $e_{u,r} \leftarrow \text{Enc}(s_{u,r})$ 
27      $t_u \leftarrow \text{Pool}(\{e_{u,r}\}_{r=1}^{R_u})$ 
28 Graph forward (once): run the last-round graph encoder on the original local TAG Obtain graph embeddings  $\{g_v\}_{v \in V^{(m)}}$  foreach
     $v \in \hat{V}_t^{(m)}$  do
29    $\tilde{\mathcal{N}}_v^{(0)} \leftarrow \{v\}$   $\tilde{\mathcal{N}}_v^{(1)} \leftarrow \mathcal{N}_v^{(1)}$   $\tilde{\mathcal{N}}_v^{(2)} \leftarrow \text{TopK}_{B_2}(\{u_w\}_{w \in \mathcal{N}_v^{(2)}})$  foreach  $\ell \in \{0, 1, 2\}$  do
30     foreach  $u \in \tilde{\mathcal{N}}_v^{(\ell)}$  do
31        $s_{v,u} \leftarrow \frac{(W_q g_v)^\top (W_k t_u)}{\sqrt{d}}$ 
32        $\tilde{\alpha}_{v,\cdot}^{(\ell)} \leftarrow \text{entmax}(\{s_{v,u}\}_{u \in \tilde{\mathcal{N}}_v^{(\ell)}})$   $\alpha_{v,\cdot}^{(\ell)} \leftarrow \Pi_{B_\ell}(\tilde{\alpha}_{v,\cdot}^{(\ell)})$   $\mathcal{S}_v^{(\ell)} \leftarrow \{u \in \tilde{\mathcal{N}}_v^{(\ell)} : \alpha_{v,u}^{(\ell)} > 0\}$ 
33    $c_v \leftarrow 0$  foreach  $\ell \in \{0, 1, 2\}$  do
34     foreach  $u \in \mathcal{S}_v^{(\ell)}$  do
35        $c_v \leftarrow c_v + \gamma_\ell \cdot \alpha_{v,u}^{(\ell)} \cdot t_u$ 
36    $\mathcal{E}_v \leftarrow \{(u, r) : u \in \cup_\ell \mathcal{S}_v^{(\ell)}, r \in [R_u]\}$   $q_v \leftarrow W_s g_v$  foreach  $(u, r) \in \mathcal{E}_v$  do
37      $a_{v,(u,r)} \leftarrow \frac{q_v^\top e_{u,r}}{\sqrt{d}}$ 
38    $\tilde{\pi}_{v,\cdot} \leftarrow \text{entmax}(\{a_{v,(u,r)}\}_{(u,r) \in \mathcal{E}_v})$   $\pi_{v,\cdot} \leftarrow \Pi_{B_{\text{tok}}}(\tilde{\pi}_{v,\cdot})$   $\tilde{t}_v \leftarrow 0$  foreach  $(u, r) \in \mathcal{E}_v$  do
39      $\tilde{t}_v \leftarrow \tilde{t}_v + \pi_{v,(u,r)} \cdot e_{u,r}$ 
40   Store neighbor cues via  $\text{TopK}(\alpha_{v,\cdot}^{(\ell)})$  Store evidence pack via  $\text{TopK}(\pi_{v,\cdot})$ 
41 Return  $\{\tilde{t}_v\}_{v \in \hat{V}_t^{(m)}}$ ,  $\{\alpha^{(\ell)}\}$ ,  $\{\pi\}$ 
    
```

Algorithm 4 Self-expressive Topology Reconstruction (per client m at round t)

Input: Core $\hat{V}_t^{(m)}$; embeddings $\{g_v\}$; evidence $\{\tilde{t}_v\}$; fusion (W_g, W_t, w) ; evidence prior S ; candidate size q ; (λ_1, λ_3) ; top- k operator $\mathcal{T}_k$; step size η ; inner iterations L

Output: Reconstructed adjacency $\hat{A}_t^{(m)}$; (optional) coefficients Z

```

42 foreach  $v \in \hat{V}_t^{(m)}$  do
43    $\alpha_v \leftarrow \sigma(w^\top [g_v \| \tilde{t}_v])$   $x_v \leftarrow \text{LN}(W_g g_v + \alpha_v \cdot W_t \tilde{t}_v)$ 
44    $X \leftarrow [x_v]_{v \in \hat{V}_t^{(m)}}$ 
45 foreach  $i \in \{1, \dots, K\}$  do
46    $C_x(i) \leftarrow \text{TopK}_q(\{x_i^\top x_j\}_{j \neq i})$   $C_S(i) \leftarrow \text{TopK}_q(\{S_{ij}\}_{j \neq i})$   $\mathcal{C}(i) \leftarrow C_x(i) \cup C_S(i)$ 
47 Initialize  $Z \leftarrow 0$  Set  $\text{diag}(Z) \leftarrow 0$  Mask  $Z_{ij} \leftarrow 0$  for all  $j \notin \mathcal{C}(i)$ 
48 for  $\ell = 1$  to  $L$  do
49    $G \leftarrow X^\top (XZ - X)$  Mask  $G_{ij} \leftarrow 0$  for all  $j \notin \mathcal{C}(i)$  Set  $G_{ii} \leftarrow 0$ 
50    $Z \leftarrow Z - \eta G$ 
51   foreach masked entry  $(i, j)$  with  $j \in \mathcal{C}(i)$  and  $i \neq j$  do
52      $\tau_{ij} \leftarrow \eta(\lambda_1 + \lambda_3(1 - S_{ij}))$   $Z_{ij} \leftarrow \text{sign}(Z_{ij}) \cdot \max(|Z_{ij}| - \tau_{ij}, 0)$ 
53   Set  $\text{diag}(Z) \leftarrow 0$  Mask  $Z_{ij} \leftarrow 0$  for all  $j \notin \mathcal{C}(i)$ 
54  $W \leftarrow |Z|$   $W \leftarrow W + |Z|^\top$   $\hat{A}_t^{(m)} \leftarrow \mathcal{T}_k(W)$ 
55 Return  $\hat{A}_t^{(m)}$  (and  $Z$ )
    
```

E. Proof of Theoretical Analysis

E.1. Budget Guarantees and Cost Decomposition

We first formalize the hard budget guarantees induced by the truncation operator Π_B used in neighbor gating and chunk selection, and then decompose the per-round client cost into (i) scoring and selection and (ii) budgeted aggregation. Throughout, $\Pi_B(\cdot)$ keeps the top- B entries (ties broken by a fixed deterministic rule) and renormalizes.

Assumption E.1 (Bounded representations). There exist constants $G, M > 0$ such that along training, $\|g_v\|_2 \leq G$ for all graph-side embeddings and $\|e_{u,r}\|_2 \leq M$ for all chunk embeddings. This can be enforced in practice via normalization or clipping.

Proposition E.2 (Hard budget guarantees). *Fix a client m and a round t . For each core node $v \in \hat{V}_t^{(m)}$:*

- (i) *For each hop $\ell \in \{0, 1, 2\}$, the selected neighbor set satisfies $|\mathcal{S}_v^{(\ell)}| \leq B_\ell$.*
- (ii) *The chunk-selection weights $\pi_{v,\cdot}$ have at most B_{tok} nonzeros, hence at most B_{tok} chunks contribute to $\tilde{t}_v$.*
- (iii) *Across the condensed core, the total number of nonzero chunk contributions to $\{\tilde{t}_v\}_{v \in \hat{V}_t^{(m)}}$ is at most $K \cdot B_{\text{tok}}$ per round.*

Proof. (i) In Eq. (5), $\alpha_{v,\cdot}^{(\ell)} = \Pi_{B_\ell}(\cdot)$ keeps at most B_ℓ nonzeros, hence $|\mathcal{S}_v^{(\ell)}| \leq B_\ell$.

(ii) In Eq. (7), $\pi_{v,\cdot} = \Pi_{B_{\text{tok}}}(\cdot)$ keeps at most B_{tok} nonzeros.

(iii) By (ii), each core node aggregates at most B_{tok} chunk embeddings in Eq. (8). Summing over $K = |\hat{V}_t^{(m)}|$ yields at most $K B_{\text{tok}}$ nonzero chunk contributions per round. $\square$

Corollary E.3 (Per-round scoring vs. aggregation cost). *Assume each node text is split into at most R_{max} chunks, i.e., $R_u \leq R_{\text{max}}$. For a core node v , the neighbor scoring in Eq. (4) costs*

$$O\left(d \sum_{\ell \in \{0,1,2\}} |\tilde{\mathcal{N}}_v^{(\ell)}|\right),$$

while the budgeted neighbor aggregation in Eq. (6) costs

$$O\left(d \sum_{\ell \in \{0,1,2\}} B_\ell\right).$$

Moreover, chunk candidates are collected only from selected neighbors, so

$$|\mathcal{E}_v| \leq \left(\sum_{\ell} |\mathcal{S}_v^{(\ell)}|\right) R_{\text{max}} \leq \left(\sum_{\ell} B_\ell\right) R_{\text{max}}.$$

Therefore, chunk scoring in Eq. (7) costs

$$O\left(d \left(\sum_{\ell} B_\ell\right) R_{\text{max}}\right),$$

while the budgeted chunk aggregation in Eq. (8) costs

$$O(d B_{\text{tok}}).$$

Summing over K core nodes yields the total per-round cost of evidence selection/aggregation on client m . In addition, the graph encoder forward pass on the original local subgraph contributes an additive backbone cost $C_{\text{GNN}}(G_m)$ per round.

Proof. Each score in Eq. (4) or Eq. (7) is an inner product in $\mathbb{R}^d$, hence $O(d)$ time. The number of neighbor scores equals $|\tilde{\mathcal{N}}_v^{(\ell)}|$ per hop, giving the stated neighbor scoring cost.

By Proposition E.2(i), at most B_ℓ neighbors per hop contribute to Eq. (6), and each contribution is a d -dimensional weighted sum, giving $O(d \sum_\ell B_\ell)$.

For chunks, $\mathcal{E}_v$ enumerates chunks only from selected neighbors. Each selected neighbor contributes at most $R_{\max}$ chunks, hence $|\mathcal{E}_v| \leq (\sum_\ell B_\ell) R_{\max}$, and chunk scoring costs $O(d(\sum_\ell B_\ell) R_{\max})$. Finally, Proposition E.2(ii) ensures at most B_{tok} chunks contribute to Eq. (8), yielding aggregation cost $O(dB_{\text{tok}})$. Summing over K core nodes gives the per-round client cost, and the backbone term $C_{\text{GNN}}(G_m)$ follows from the required forward pass producing $\{g_v\}$. $\square$

The above bounds capture the dominant arithmetic cost from inner products and budgeted weighted sums. Selection operators such as $\text{entmax}(\cdot)$ and top- B truncation can incur additional $O(n \log n)$ (or $O(n \log B)$) time for sorting/thresholding over n candidates; these terms are standard and omitted from the main expressions for clarity.

E.2. Approximation Error from Hard Budget Projection

We quantify the approximation error introduced solely by the hard truncation $\Pi_{B_{\text{tok}}}$ in Eq. (7), relative to using the *same* scoring rule without truncation (i.e., only removing the top- B_{tok} renormalization step).

Definition E.4 (Tail mass under top- B truncation). For a probability vector $p \in \Delta^n$, let $T_B(p)$ denote the indices of its top- B entries (ties broken by a fixed rule), and define the tail mass

$$\delta_B(p) \triangleq \sum_{i \notin T_B(p)} p_i.$$

Lemma E.5 (Truncation bound for probability-weighted sums). Let $\{e_i\}_{i=1}^n \subset \mathbb{R}^d$ satisfy $\|e_i\|_2 \leq M$. For any $p \in \Delta^n$ and $\tilde{p} = \Pi_B(p)$,

$$\left\| \sum_{i=1}^n p_i e_i - \sum_{i=1}^n \tilde{p}_i e_i \right\|_2 \leq 2M \cdot \delta_B(p).$$

Proof. Let $T = T_B(p)$ and $s = \sum_{i \in T} p_i = 1 - \delta_B(p)$. By definition of Π_B , $\tilde{p}_i = \frac{p_i}{s}$ for $i \in T$ and $\tilde{p}_i = 0$ otherwise, hence $\sum_{i=1}^n \tilde{p}_i e_i = \sum_{i \in T} \frac{p_i}{s} e_i$. Decompose the difference into discarded mass and renormalization bias:

$$\sum_{i=1}^n p_i e_i - \sum_{i \in T} \frac{p_i}{s} e_i = \underbrace{\sum_{i \notin T} p_i e_i}_{(A)} + \underbrace{\sum_{i \in T} p_i \left(1 - \frac{1}{s}\right) e_i}_{(B)}.$$

For term (A), $\|(A)\|_2 \leq \sum_{i \notin T} p_i \|e_i\|_2 \leq M \delta_B(p)$. For term (B), note that $\left|1 - \frac{1}{s}\right| = \frac{1-s}{s} = \frac{\delta_B(p)}{1-\delta_B(p)}$ and

$$\|(B)\|_2 \leq \left(\frac{1}{s} - 1\right) \sum_{i \in T} p_i \|e_i\|_2 \leq \left(\frac{1}{s} - 1\right) Ms = M(1-s) = M \delta_B(p).$$

Combining both bounds yields the result. $\square$

Proposition E.6 (Chunk condensation approximation error in DANCE). Fix a core node v and its candidate chunk set $\mathcal{E}_v$. Let

$$p_v \triangleq \text{entmax}(\{a_{v,(u,r)}\}_{(u,r) \in \mathcal{E}_v}) \in \Delta^{|\mathcal{E}_v|}$$

be the (untruncated) probability vector produced by entmax , and let $\pi_v = \Pi_{B_{\text{tok}}}(p_v)$ be the budgeted weights in Eq. (7). Define the unbudgeted and budgeted evidence embeddings

$$t_v^{\text{full}} \triangleq \sum_{(u,r) \in \mathcal{E}_v} (p_v)_{(u,r)} e_{u,r}, \quad \tilde{t}_v \triangleq \sum_{(u,r) \in \mathcal{E}_v} (\pi_v)_{(u,r)} e_{u,r}.$$

Under Assumption E.1,

$$\|\tilde{t}_v - t_v^{\text{full}}\|_2 \leq 2M \cdot \delta_{B_{\text{tok}}}(p_v).$$

An analogous bound holds for neighbor gating at hop ℓ by replacing p_v with the corresponding (untruncated) neighbor-weight vector and replacing B_{tok} with B_ℓ .

Proof. Apply Lemma E.5 to the collection $\{e_{u,r}\}_{(u,r) \in \mathcal{E}_v}$ with $p = p_v$ and $\tilde{p} = \Pi_{B_{\text{tok}}}(p_v) = \pi_v$. $\square$

Remark. The error scales with the discarded probability mass $\delta_{B_{\text{tok}}}(p_v)$. When entmax already produces a sparse distribution (small tail mass), the additional hard truncation incurs a correspondingly small approximation error.

E.3. Stability of Model-in-the-loop Evidence Refresh

Finally, we characterize when the round-wise refresh changes the selected neighbors/chunks. DANCE forms selection weights by applying $\text{entmax}(\cdot)$ to the scores and then performing hard top- B truncation via Π_B . Since both steps preserve the score ordering (up to ties), the selected index set is stable as long as the score ranking around the top- B boundary does not change.

Definition E.7 (Top- B index set). For a vector $r \in \mathbb{R}^n$, let $T_B(r)$ denote the indices of its top- B entries (ties broken by a fixed rule).

Lemma E.8 (Order preservation implies identical top- B sets). Let $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ be strictly increasing and apply it elementwise to $r \in \mathbb{R}^n$. Then for any B , $T_B(r) = T_B(\varphi(r))$.

Proof. For any i, j , strict monotonicity gives $r_i > r_j \Leftrightarrow \varphi(r_i) > \varphi(r_j)$, so the ordering (and hence the top- B set) is unchanged. $\square$

Assumption E.9 (Local Lipschitzness of scores along the trajectory). Along the parameter trajectory $\{\omega^{(t)}\}_{t=0}^T$, for any fixed node v and any candidate index j (neighbor or chunk), the corresponding score function satisfies

$$|\text{score}_j(\omega) - \text{score}_j(\omega')| \leq L_s \|\omega - \omega'\|_2 \quad \text{for all } \omega, \omega' \text{ on the trajectory.}$$

A sufficient condition is $\|\nabla_{\omega} \text{score}_j(\omega)\|_2 \leq L_s$ along the trajectory (e.g., via weight decay/clipping and Assumption E.1).

Definition E.10 (Top- B margin). Let $r(\omega) \in \mathbb{R}^n$ be candidate scores sorted as $r_{(1)}(\omega) \geq \dots \geq r_{(n)}(\omega)$. Define the margin

$$\Delta_B(\omega) \triangleq r_{(B)}(\omega) - r_{(B+1)}(\omega).$$

Theorem E.11 (Stability of DANCE evidence selection across rounds). Fix a node v and consider either neighbor selection at hop ℓ (budget B_ℓ) or chunk selection (budget B_{tok}). Let $r(\omega)$ denote the corresponding pre-entmax score vector (i.e., $s_{v,u}$ or $a_{v,(u,r)}$). Under Assumption E.9, if

$$\|\omega - \omega'\|_2 \leq \frac{\Delta_B(\omega)}{2L_s},$$

then the selected index set after entmax and truncation is invariant:

$$T_B\left(\Pi_B(\text{entmax}(r(\omega)))\right) = T_B\left(\Pi_B(\text{entmax}(r(\omega')))\right),$$

where B is B_ℓ or B_{tok} accordingly.

Proof. By Assumption E.9, for every coordinate j , $|r_j(\omega) - r_j(\omega')| \leq L_s \|\omega - \omega'\|_2$. If $\|\omega - \omega'\|_2 \leq \Delta_B(\omega)/(2L_s)$, then the standard ranking-stability argument implies $T_B(r(\omega)) = T_B(r(\omega'))$, since no element below rank B can overtake the B -th element (and vice versa) under such bounded perturbations.

Next, since the post-entmax truncation Π_B selects indices based on the entmax output, the selected set remains invariant provided that entmax does not change the membership of the top- B indices under the considered perturbation. This condition holds whenever the entmax support around the B -th boundary is unchanged.

Finally, $\Pi_B(\cdot)$ keeps exactly the top- B indices of its input by definition, so applying Π_B does not change the top- B index set. Combining the above yields the claim. $\square$

Theorem E.11 formalizes *controlled adaptivity* of model-in-the-loop refresh: the selected neighbors/chunks remain stable under small global-model drift when the top- B margin is non-trivial, and updates occur only when the score ranking near the top- B boundary changes.